KU LEUVEN

# Lecture 6: Evaluation

Machine learning and inductive inference

Sieben Bocklandt

*Slides based on slidedecks from Jesse Davis' lectures in Principles of Machine Learning*

# Goals of this lecture

- Metrics
  - Classification, Regression, Probability estimation
  - Understand strengths and weaknesses of metrics and know how to map them to a real problem
  - Be able to construct ROC and PR curves and understand what they show
  - Know the metrics and their behaviour by heart

# Goals of this lecture

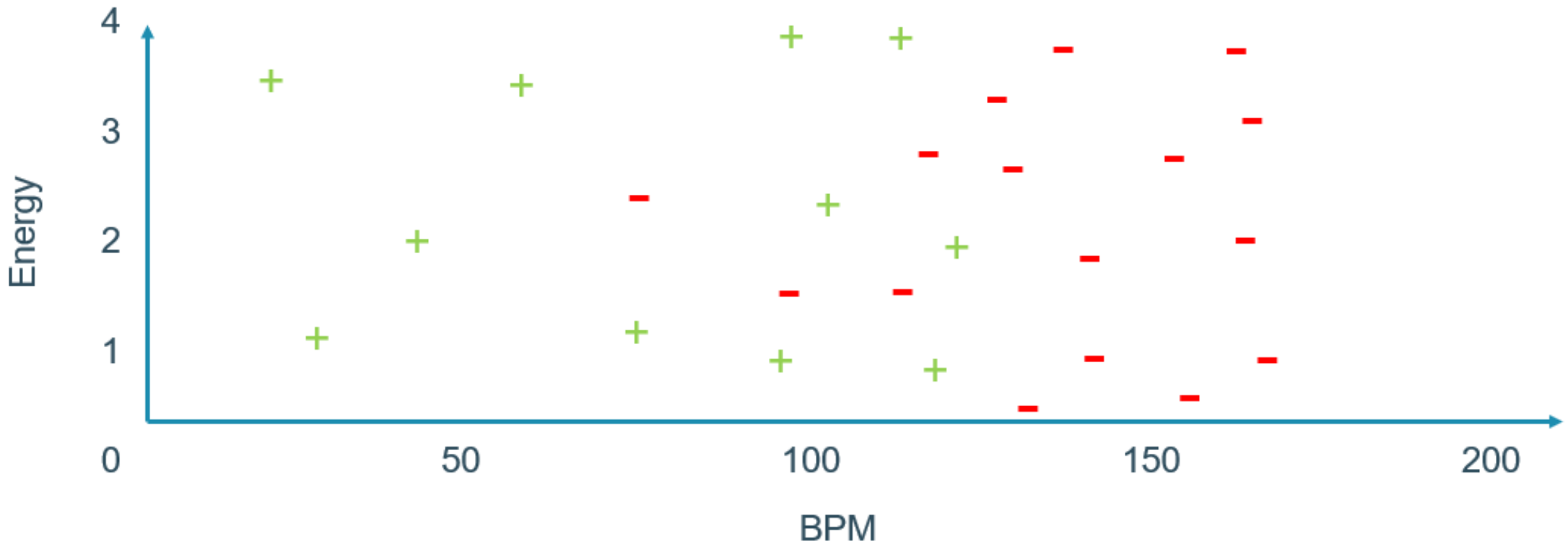
- Metrics
  - Classification, Regression, Probability estimation
  - Understand strengths and weaknesses of metrics and know how to map them to a real problem
  - Be able to construct ROC and PR curves and understand what they show
  - Know the metrics and their behaviour by heart

- Methodology
  - Know how to perform cross validation
  - Understand why we need independent training-validation-test sets
  - Know standard feature transformation and feature selection techniques

# Evaluation metrics

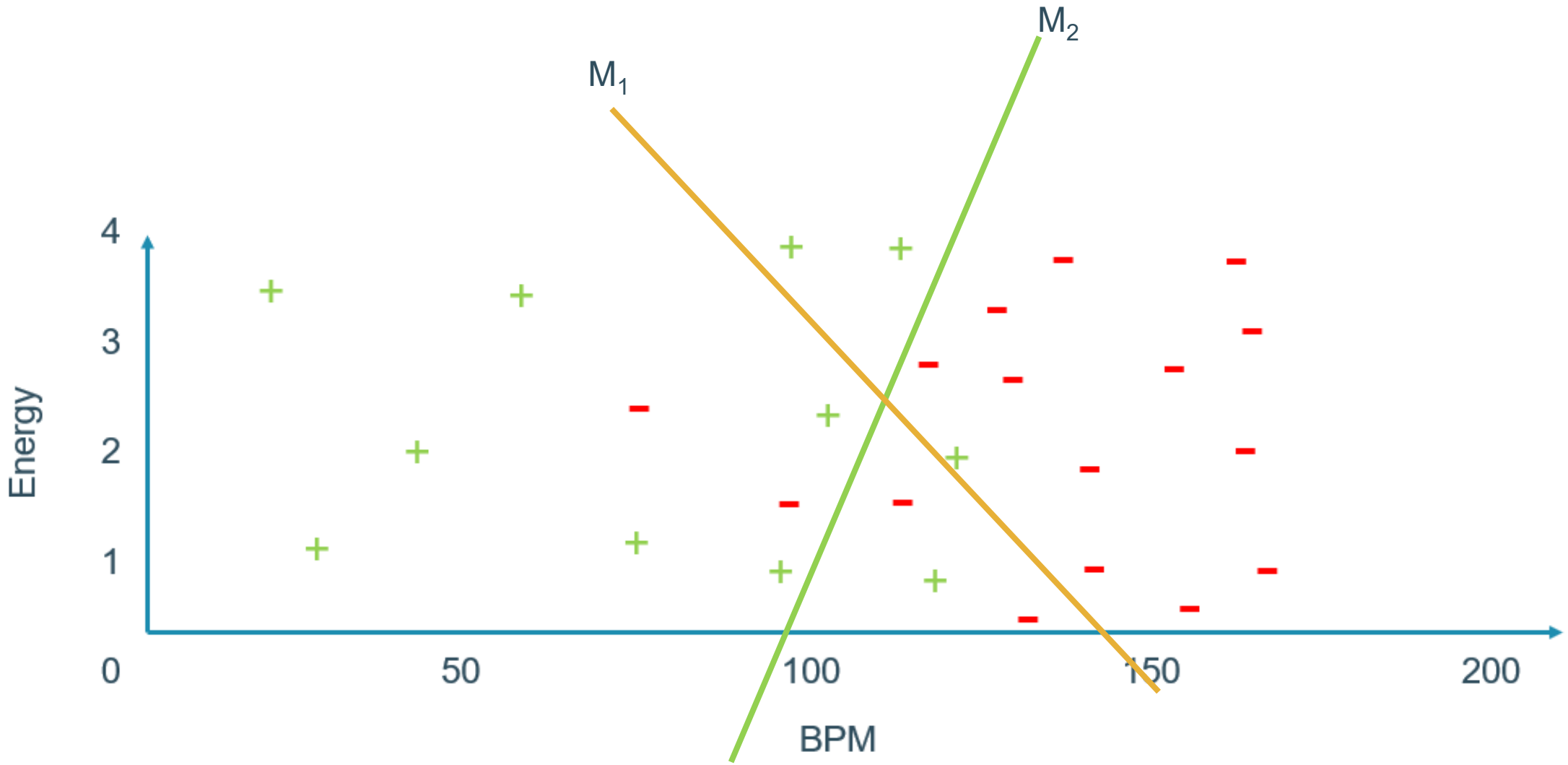


KU LEUVEN



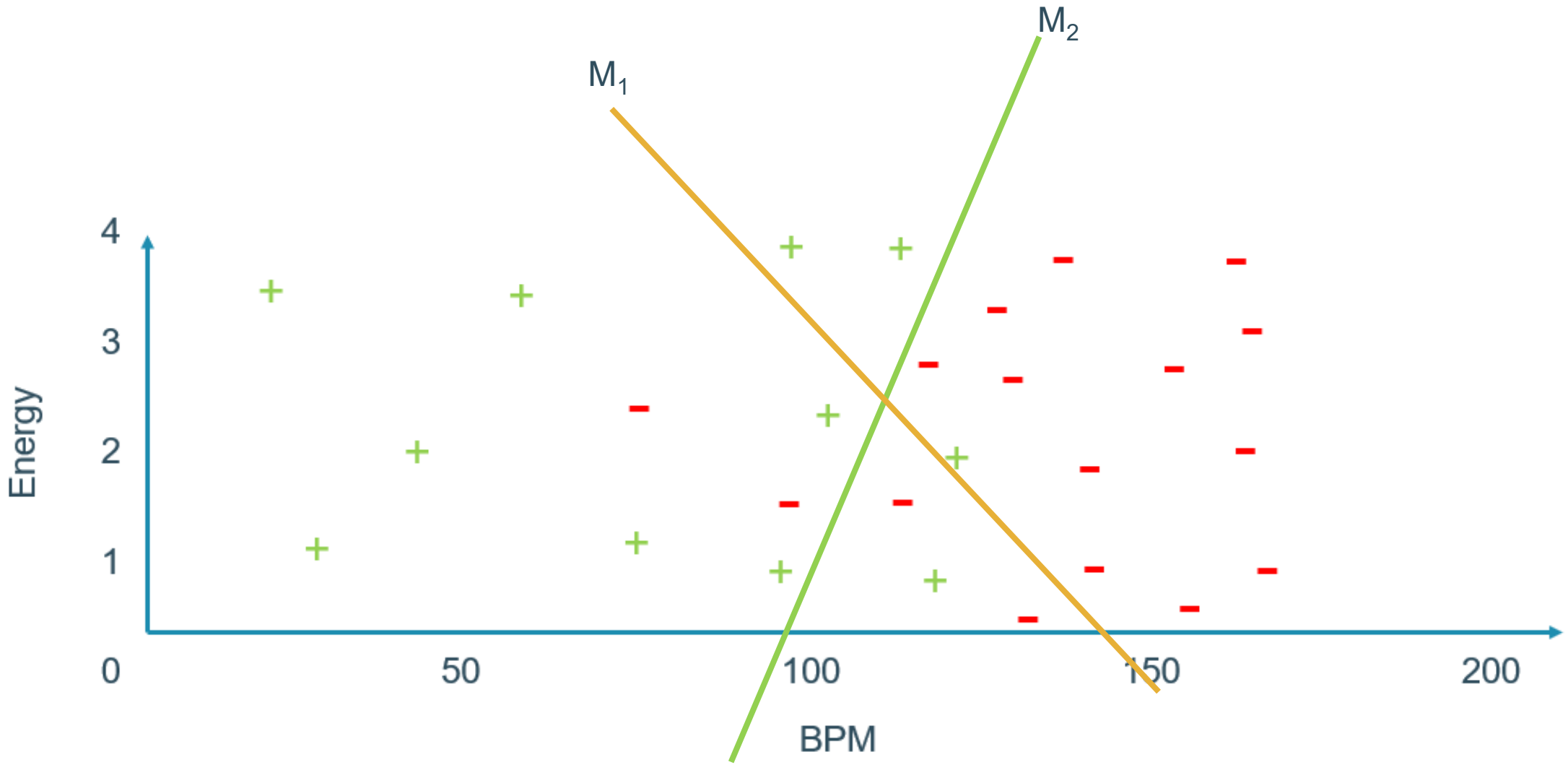



KU LEUVEN





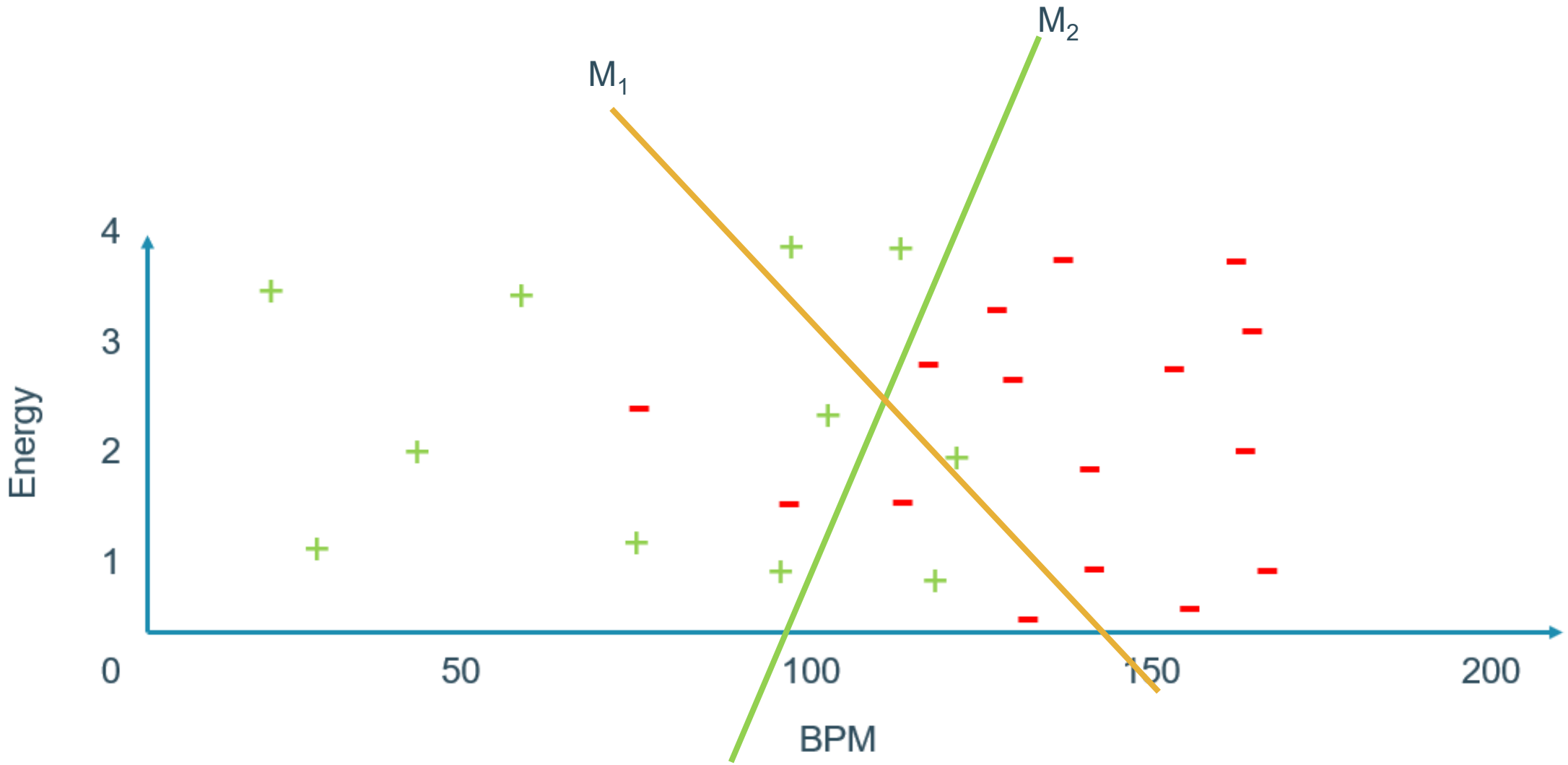

KU LEUVEN




Which model is better?



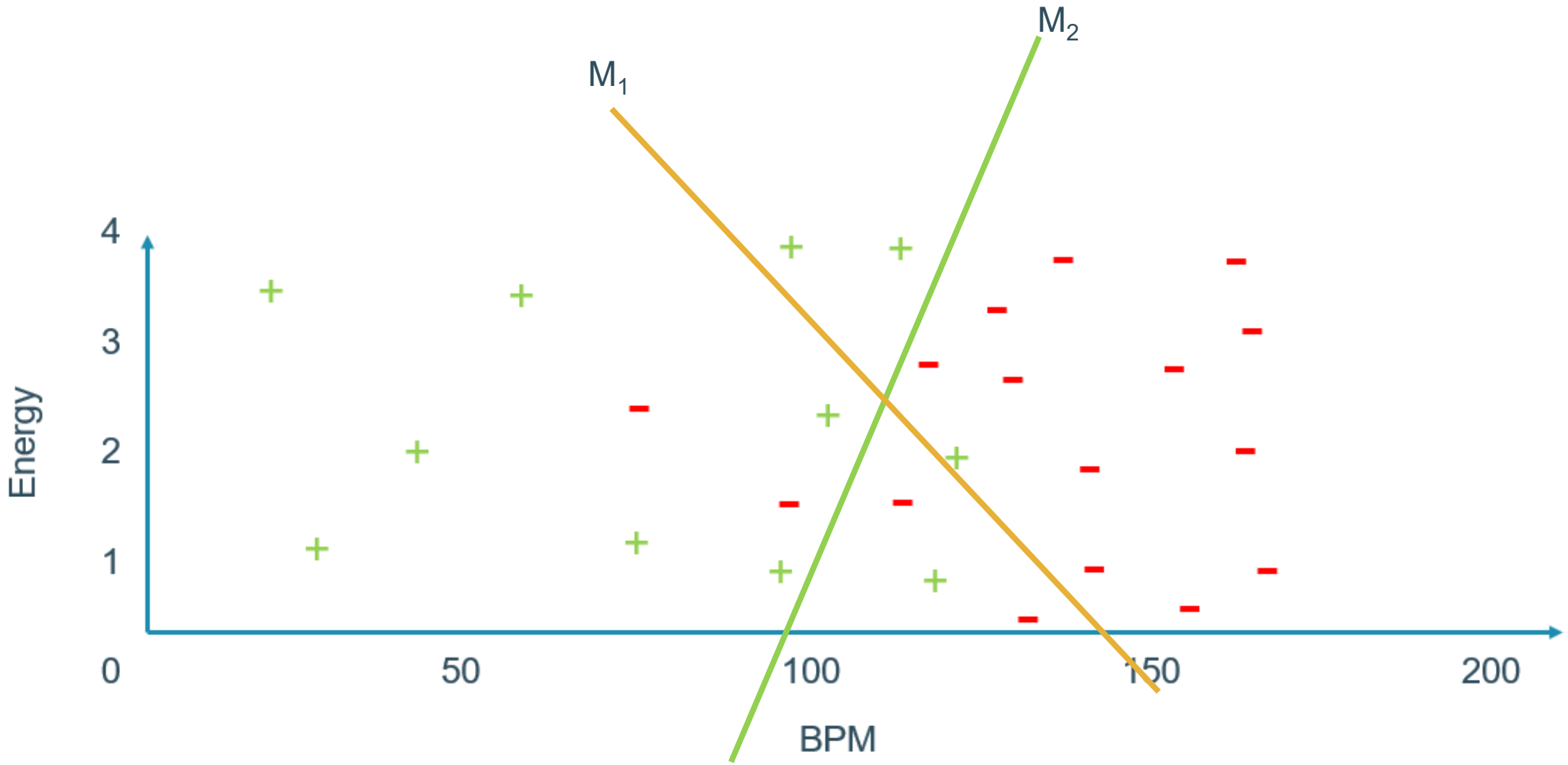
KU LEUVEN




Which model is better?
How good is each model?



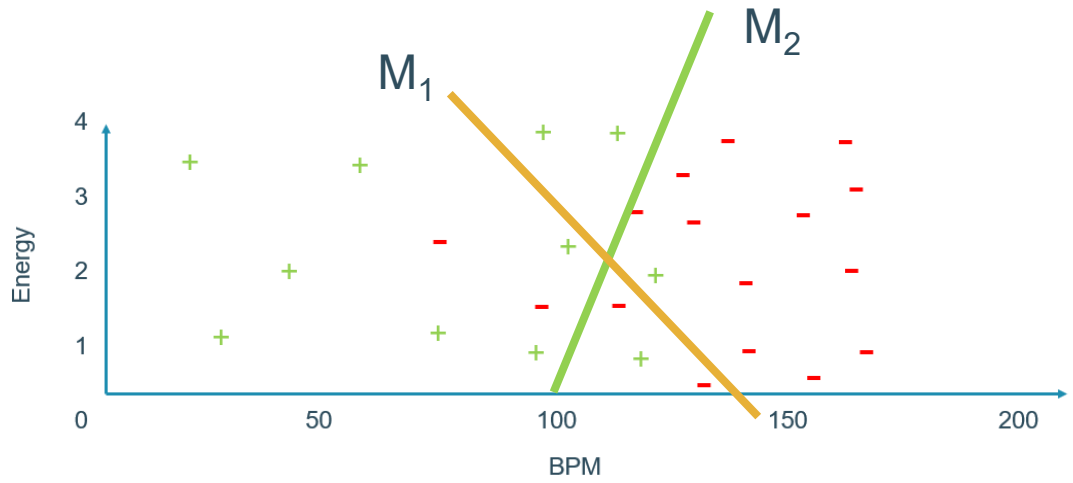
KU LEUVEN




Which model is better?
How good is each model? → How do we measure performance?

# Accuracy and Error

- Two natural performance measures for classification: Error rate and accuracy
  - Success: instance's class is predicted correctly
  - Error: instance's class is predicted incorrectly



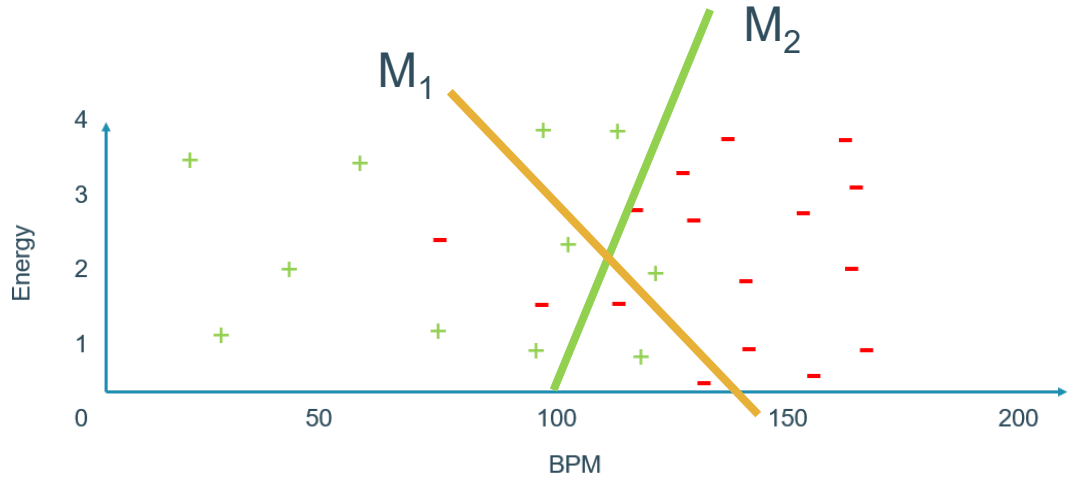



KU LEUVEN

# Accuracy and Error

- Two natural performance measures for classification: Error rate and accuracy
  - Success: instance's class is predicted correctly
  - Error: instance's class is predicted incorrectly

$$Accuracy = \frac{\#successes}{\#instances}$$



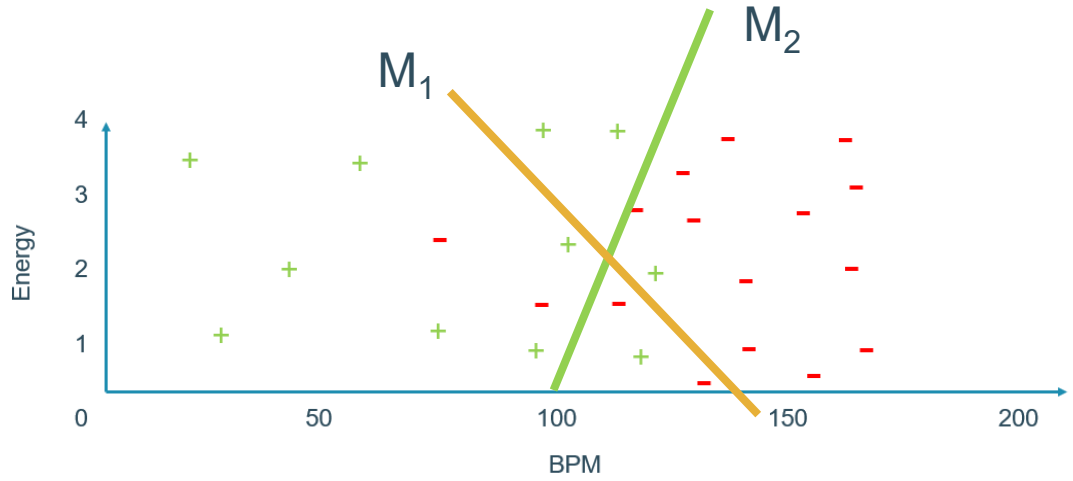



KU LEUVEN

# Accuracy and Error

- Two natural performance measures for classification: Error rate and accuracy
  - Success: instance's class is predicted correctly
  - Error: instance's class is predicted incorrectly

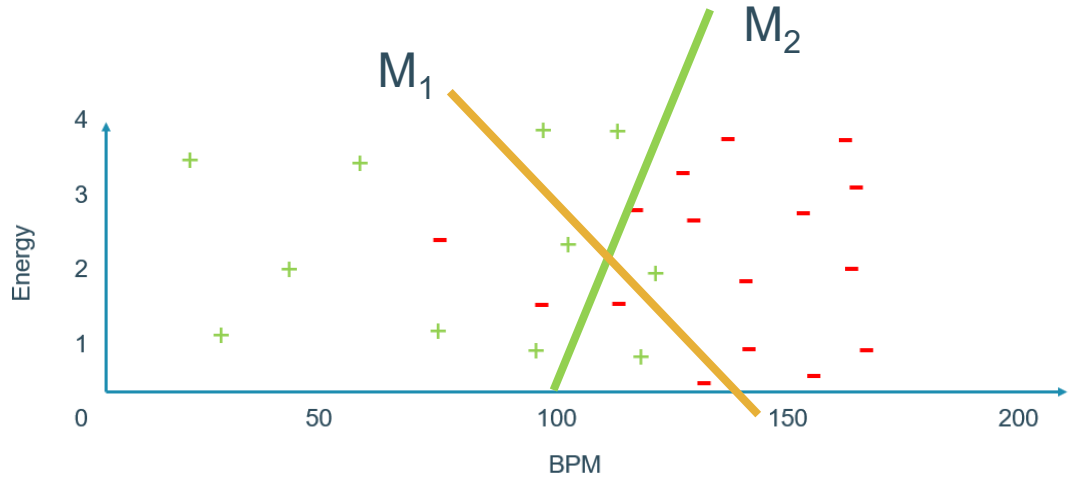
$$Accuracy = \frac{\#successes}{\#instances} \qquad Error\ rate = \frac{\#errors}{\#instances}$$



# Accuracy and Error

- Two natural performance measures for classification: Error rate and accuracy
  - Success: instance's class is predicted correctly
  - Error: instance's class is predicted incorrectly

$$Accuracy = \frac{\#successes}{\#instances} \qquad Error\ rate = \frac{\#errors}{\#instances} \qquad Accuracy = 1 - Error\ rate$$



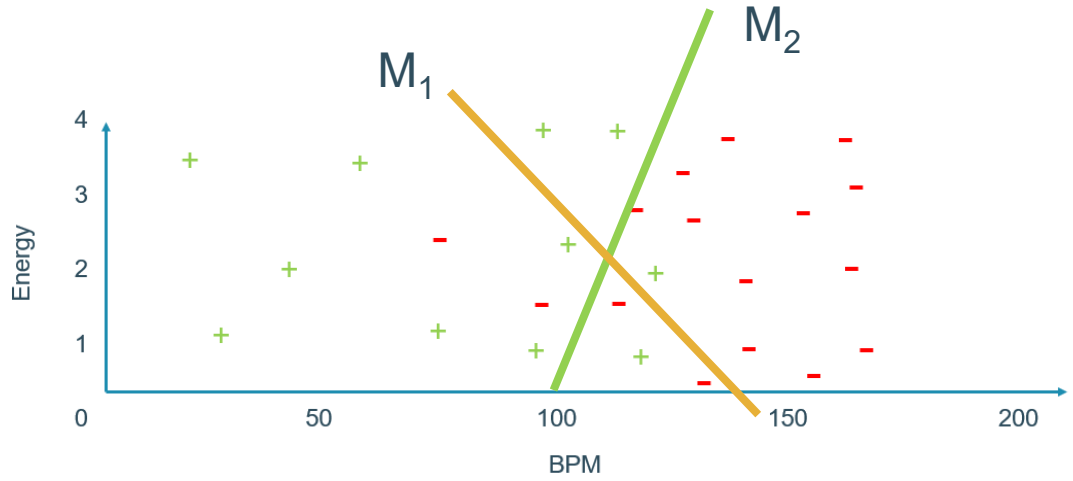



KU LEUVEN

# Accuracy and Error

- Two natural performance measures for classification: Error rate and accuracy
  - Success: instance's class is predicted correctly
  - Error: instance's class is predicted incorrectly

$$Accuracy = \frac{\#successes}{\#instances} \qquad Error\ rate = \frac{\#errors}{\#instances} \qquad Accuracy = 1 - Error\ rate$$



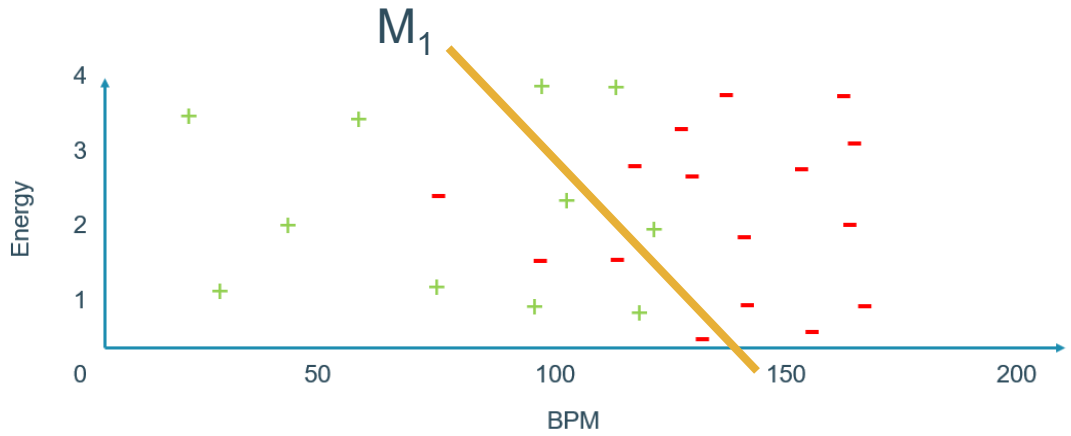

Which model has a higher accuracy?



KU LEUVEN

# Predictions explained: Confusion matrix or Contingency table for Binary Classification

| | Predicted True | Predicted False |
|---|---|---|
| Actually True | | |
| Actually False | | |

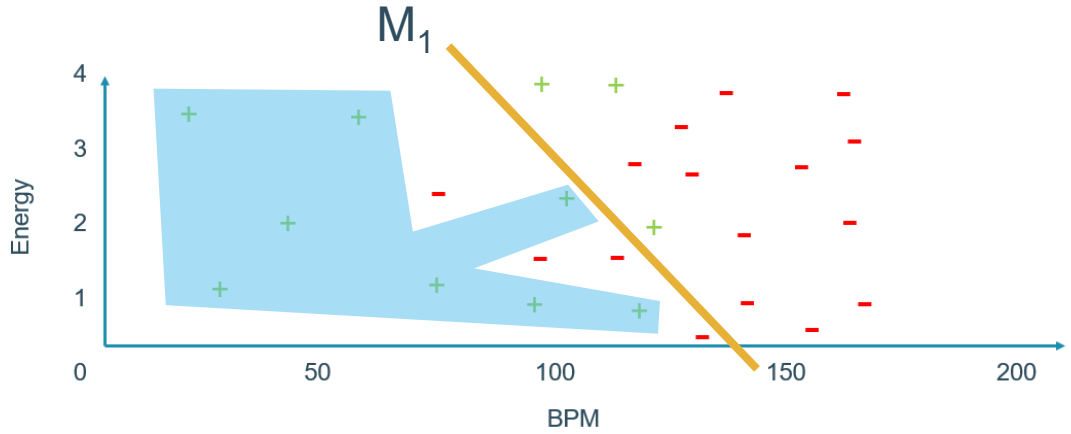





KU LEUVEN

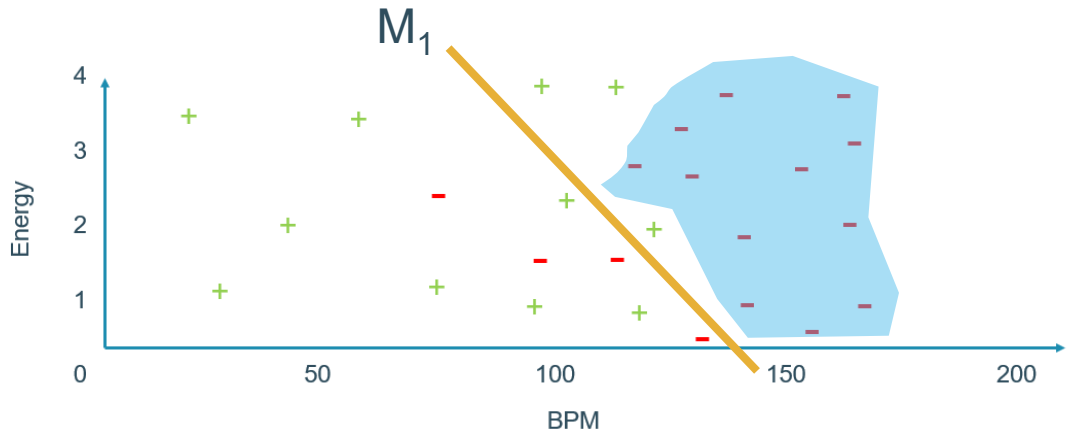
# Predictions explained: Confusion matrix or Contingency table for Binary Classification

| | Predicted True | Predicted False |
|---|---|---|
| **Actually True** | True Positive (TP)<br>8 | |
| **Actually False** | | |



# Predictions explained: Confusion matrix or Contingency table for Binary Classification

| | Predicted True | Predicted False |
|---|---|---|
| **Actually True** | True Positive (TP) 8 | |
| **Actually False** | | True Negative (TN) 12 |

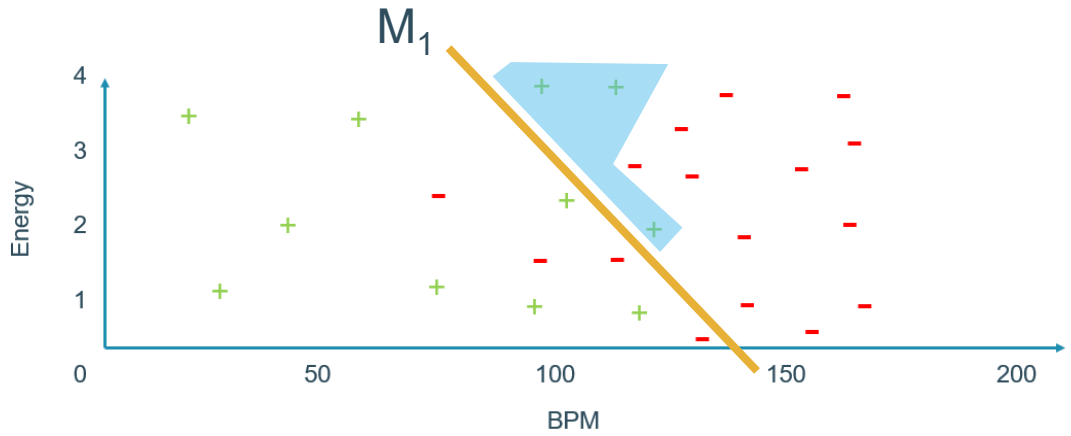





KU LEUVEN

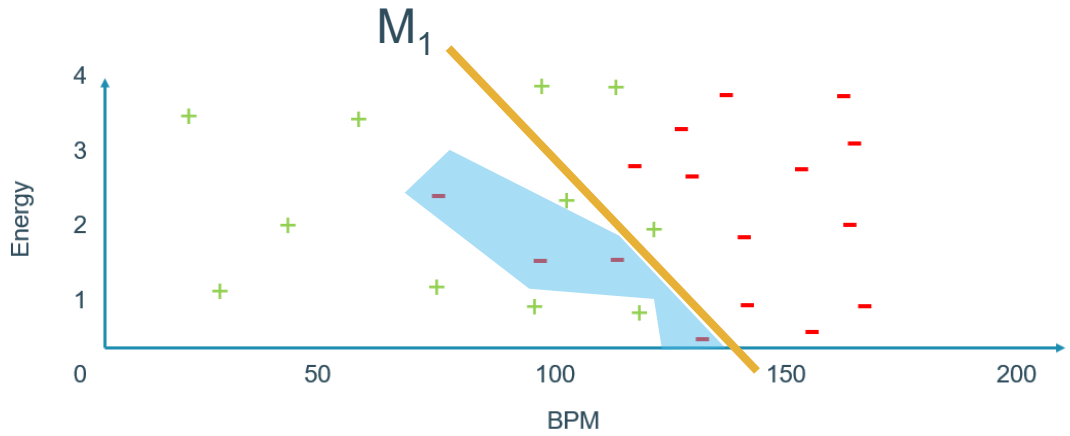
# Predictions explained: Confusion matrix or Contingency table for Binary Classification

| | Predicted True | Predicted False |
|---|---|---|
| **Actually True** | True Positive (TP)<br>8 | False Negative (FN)<br>3 |
| **Actually False** | | True Negative (TN)<br>12 |



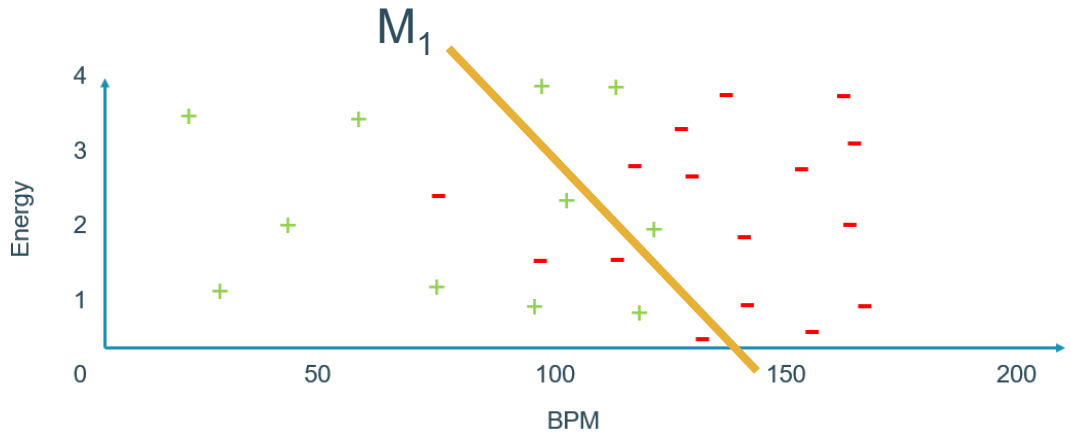
# Predictions explained: Confusion matrix or Contingency table for Binary Classification

| | Predicted True | Predicted False |
|---|---|---|
| **Actually True** | True Positive (TP)<br>8 | False Negative (FN)<br>3 |
| **Actually False** | False Positive (FP)<br>4 | True Negative (TN)<br>12 |



# Predictions explained: Confusion matrix or Contingency table for Binary Classification

| | Predicted True | Predicted False |
|---|---|---|
| **Actually True** | True Positive (TP)<br>8 | False Negative (FN)<br>3 |
| **Actually False** | False Positive (FP)<br>4 | True Negative (TN)<br>12 |

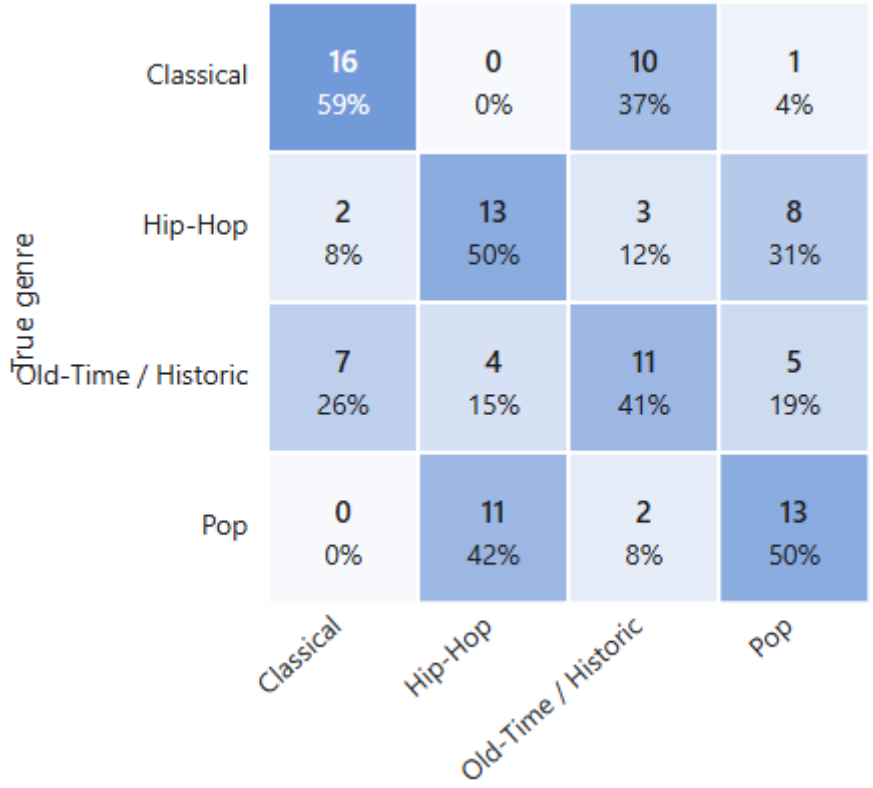



$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad Error\ rate = \frac{FP+FN}{TP+TN+FP+FN} \quad Accuracy = 1 - Error\ rate$$

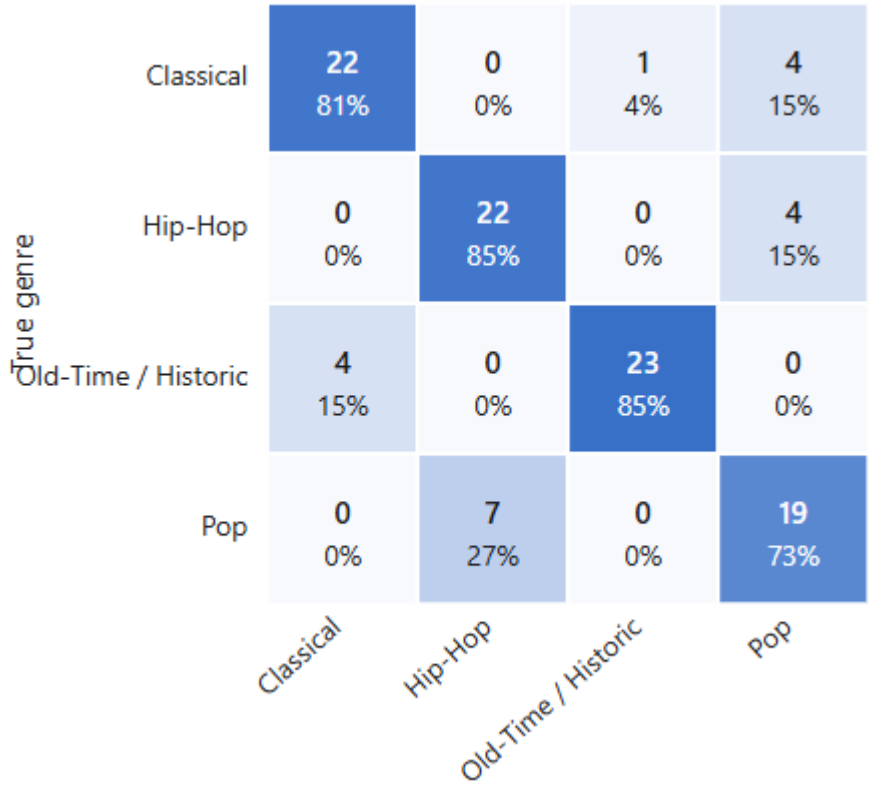


KU LEUVEN

# Multiclass confusion matrix




Confusion matrix for genre prediction (3-NN)




Confusion matrix for genre prediction (XGBoost)



KU LEUVEN

# Accuracy is sensitive to class skew

- Let class skew be $\pi = \frac{|\{x_i \mid y_i = 1\}|}{N}$
- Define true positive rate $TPR = \frac{TP}{TP+FN}$ and true negative rate $TNR = \frac{TN}{TN+FP}$
- Accuracy = weighted average of $TPR$ and $TNR$:

$$Acc = \pi \times TPR + (1 - \pi) \times TNR$$ ➔ Verify this at home!



KU LEUVEN

# Accuracy is sensitive to class skew

– Let class skew be $\pi = \frac{|\{x_i \mid y_i = 1\}|}{N}$

– Define true positive rate $TPR = \frac{TP}{TP+FN}$ and true negative rate $TNR = \frac{TN}{TN+FP}$

– Accuracy = weighted average of $TPR$ and $TNR$:

$$Acc = \pi \times TPR + (1 - \pi) \times TNR$$ ➔ Verify this at home!

– Let $TPR = 0.6$ and $TNR = 0.8$

If $\pi = 0.5$, then acc = 0.7

If $\pi = 0.9$, then acc = 0.62

– Balanced accuracy often used in research (setting $\pi = 0.5$ independent of the actual skew)

# Problems with accuracy

- Is 99% accuracy good?



KU LEUVEN

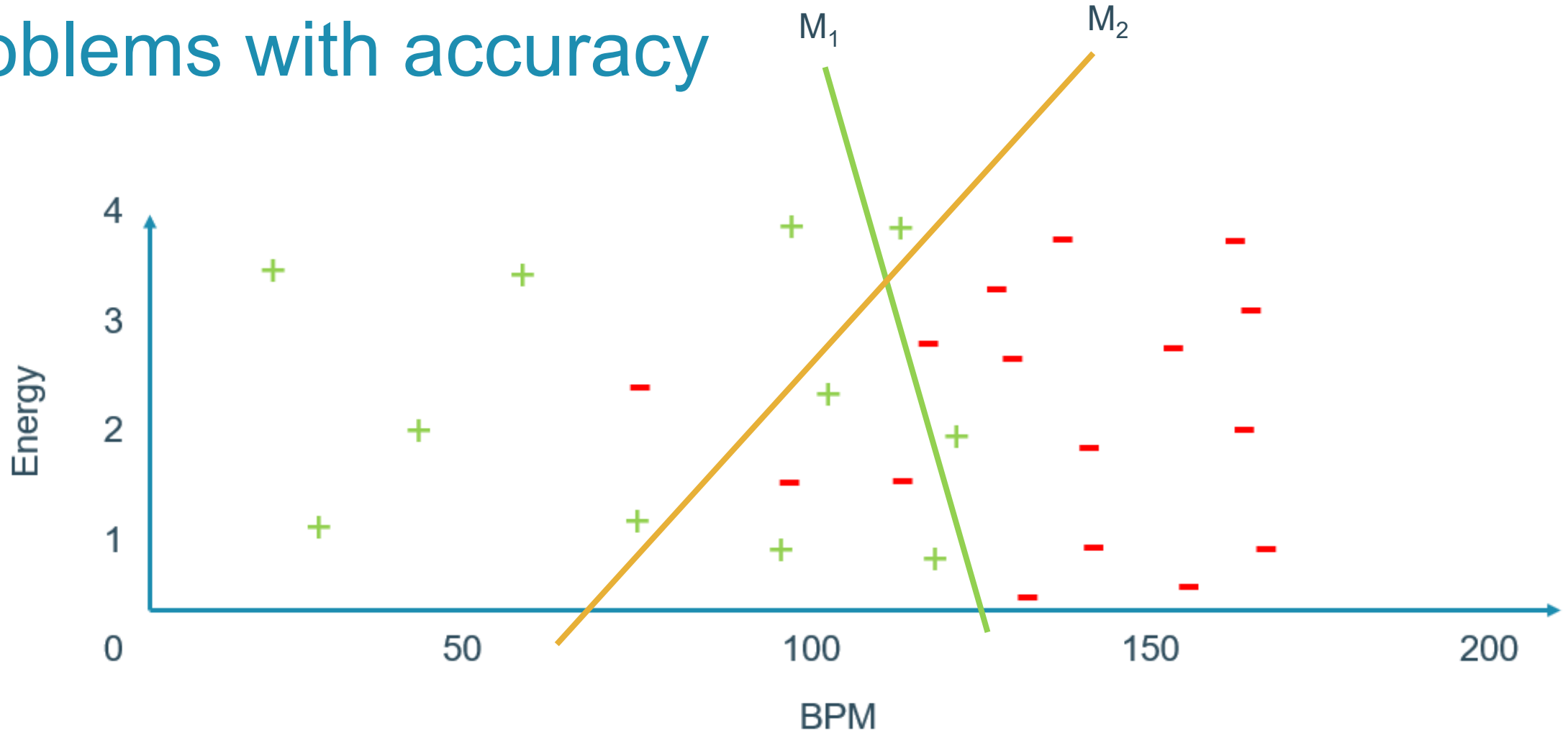
# Problems with accuracy

– Is 99% accuracy good?

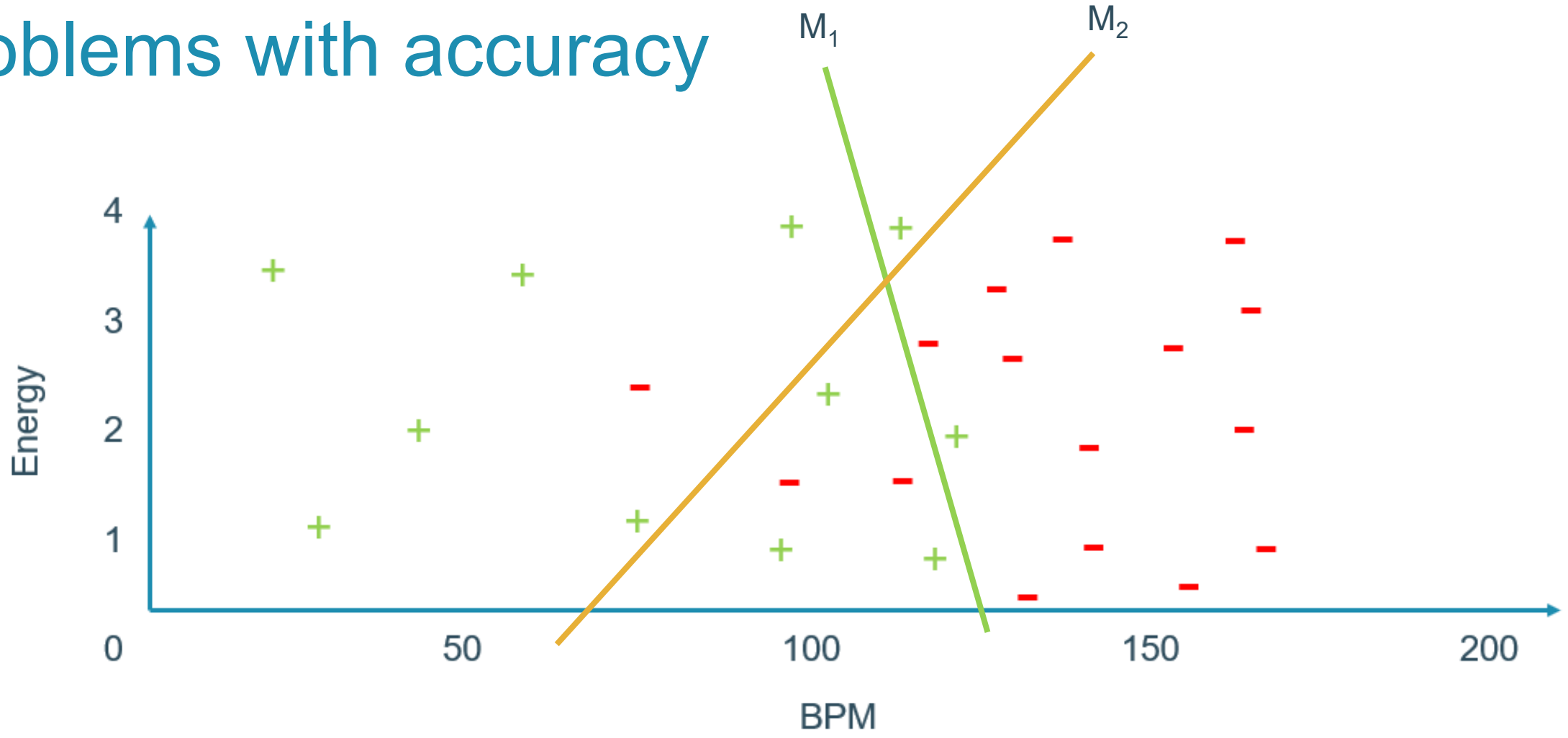
| | Predicted True | Predicted False |
|---|---|---|
| **Actually True** | 990 | 10 |
| **Actually False** | 10 | 990 |

| | Predicted True | Predicted False |
|---|---|---|
| **Actually True** | 0 | 20 |
| **Actually False** | 0 | 1980 |

# Problems with accuracy



# Problems with accuracy




Both have the same accuracy, but in some settings, one is preferred over the other.



KU LEUVEN

# Incorporating Costs

- Suppose we have misclassification costs
  - $C_{FP}$: Cost of a false positive
  - $C_{FN}$: Cost of a false negative

# Incorporating Costs

- Suppose we have misclassification costs
  - $C_{FP}$: Cost of a false positive
  - $C_{FN}$: Cost of a false negative
- Expected cost of one prediction

$$C = C_{FN} \times \pi \times P(h(x) = 0 | y = 1) + C_{FP} \times (1 - \pi) \times P(h(x) = 1 | y = 0)$$

# Incorporating Costs

- Suppose we have misclassification costs
  - $C_{FP}$: Cost of a false positive
  - $C_{FN}$: Cost of a false negative
- Expected cost of one prediction

$$C = C_{FN} \times \pi \times P(h(x) = 0|y = 1) + C_{FP} \times (1 - \pi) \times P(h(x) = 1|y = 0)$$

- We approximate this by substituting with FPR and FNR

$$C = C_{FN} \times \pi \times \boldsymbol{FNR} + C_{FP} \times (1 - \pi) \times \boldsymbol{FPR}$$

where $FNR = \frac{FN}{FN+TP}$ and $FPR = \frac{FP}{TN+FP}$



KU LEUVEN

# Incorporating Costs

- Suppose we have misclassification costs
  - $C_{FP}$: Cost of a false positive
  - $C_{FN}$: Cost of a false negative
- Expected cost of one prediction

$$C = C_{FN} \times \pi \times P(h(x) = 0|y = 1) + C_{FP} \times (1 - \pi) \times P(h(x) = 1|y = 0)$$

- We approximate this by substituting with FPR and FNR

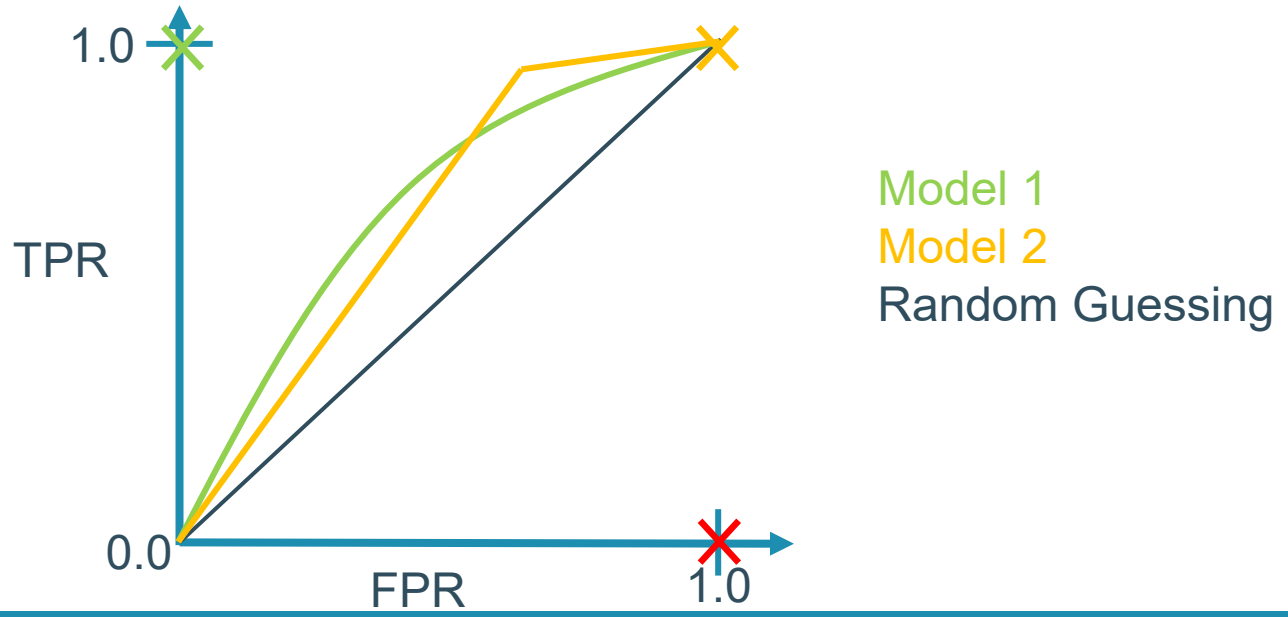
$$C = C_{FN} \times \pi \times \boldsymbol{FNR} + C_{FP} \times (1 - \pi) \times \boldsymbol{FPR}$$

where $FNR = \frac{FN}{FN+TP}$ and $FPR = \frac{FP}{TN+FP}$

- $C$ is not computable from accuracy alone

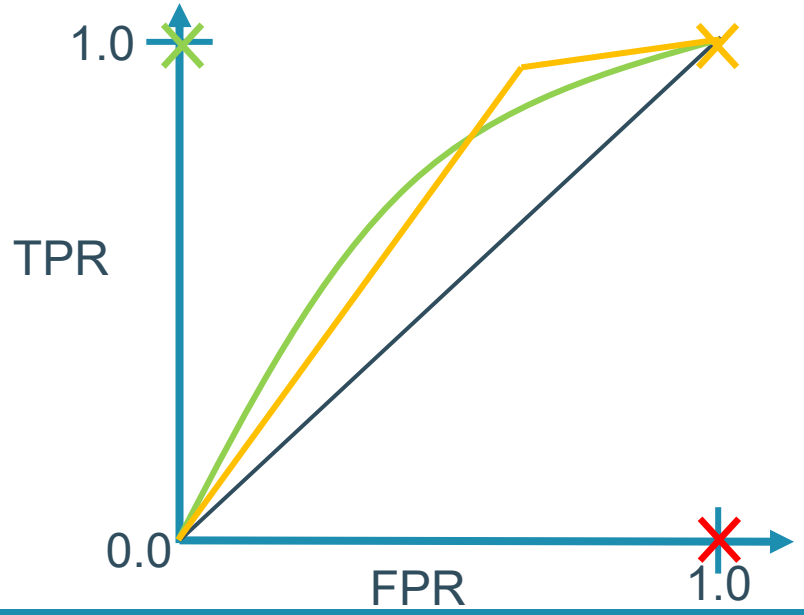
# Overcoming these issues: ROC curves

- Receiver Operating Characteristic curves
- Started during research on radars during WWII
- Works for binary classification problems
- ROC curves visually show two aspects of performance by plotting FPR and TPR



# Overcoming these issues: ROC curves

- Receiver Operating Characteristic curves
- Started during research on radars during WWII
- Works for binary classification problems
- ROC curves visually show two aspects of performance by plotting FPR and TPR




Model 1
Model 2
Random Guessing

What is shown by ✕ , ✕ and ✕?



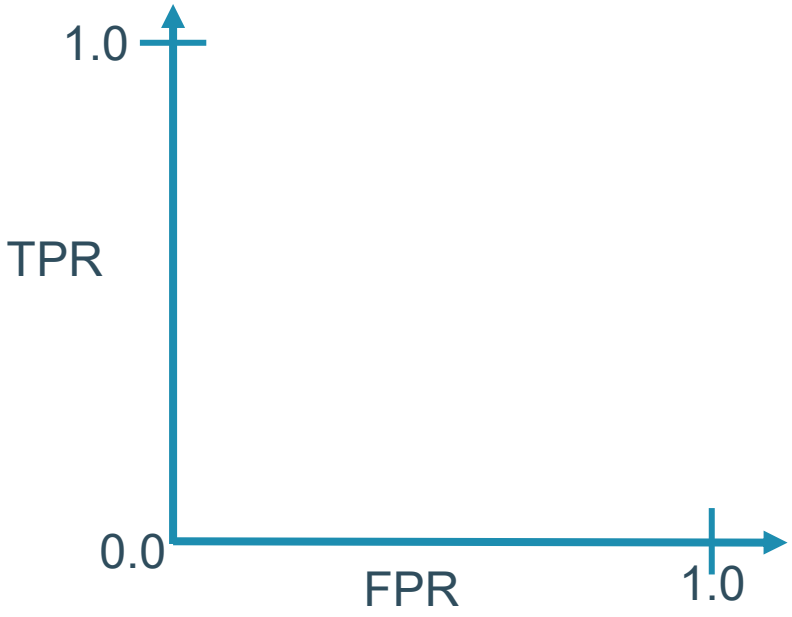
KU LEUVEN

# Constructing a ROC curve

- Given: numeric outputs for each test example (e.g. $P(y = 1|x)$)
- Approach
  1. Sort output values for test set
  2. Locate boundaries between examples with different labels
  3. Compute the FPR and TPR for each of these boundaries
  4. Plot each (FPR,TPR) point
  5. Connect the points to construct the curve

# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

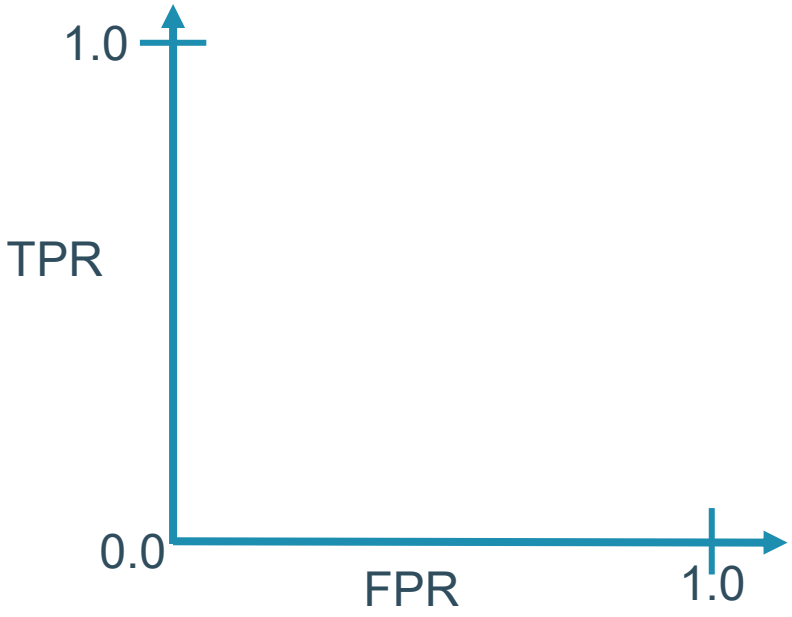


# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

→

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.93 | + |
| 0.73 | + |
| 0.63 | - |
| 0.61 | - |
| 0.52 | + |
| 0.37 | - |
| 0.33 | + |
| 0.12 | - |
| 0.05 | - |



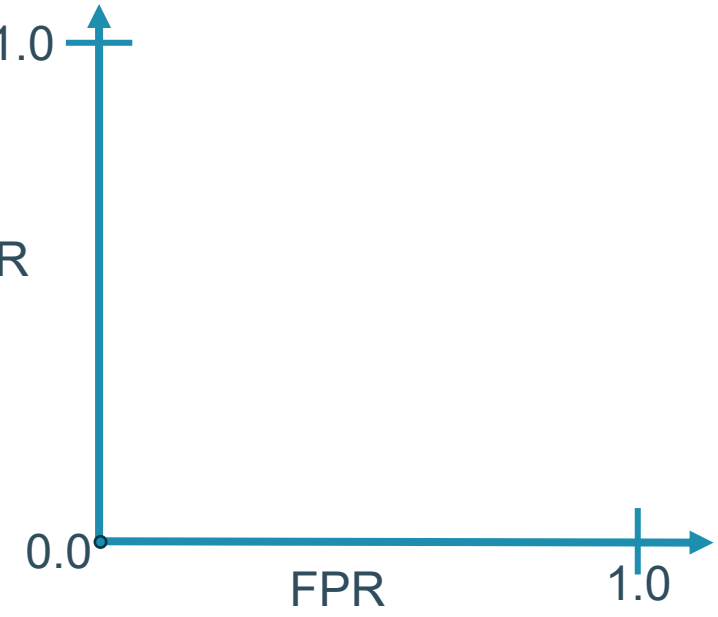
# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

→

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.93 | + |
| 0.73 | + |
| 0.63 | - |
| 0.61 | - |
| 0.52 | + |
| 0.37 | - |
| 0.33 | + |
| 0.12 | - |
| 0.05 | - |

FPR= 0.0, TPR= 0.0



# ROC: example

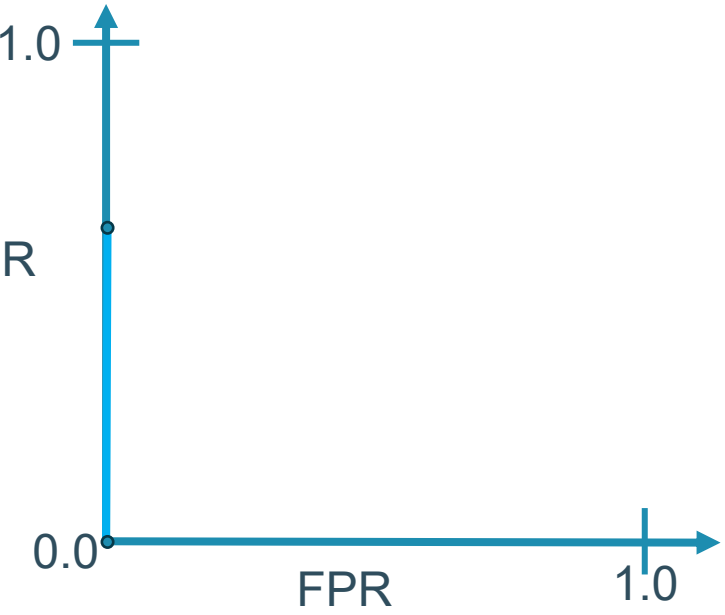
| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

→

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.93 | + |
| 0.73 | + |
| 0.63 | - |
| 0.61 | - |
| 0.52 | + |
| 0.37 | - |
| 0.33 | + |
| 0.12 | - |
| 0.05 | - |

FPR= 0.0, TPR= 0.0

FPR= 0.0, TPR= 0.6



# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

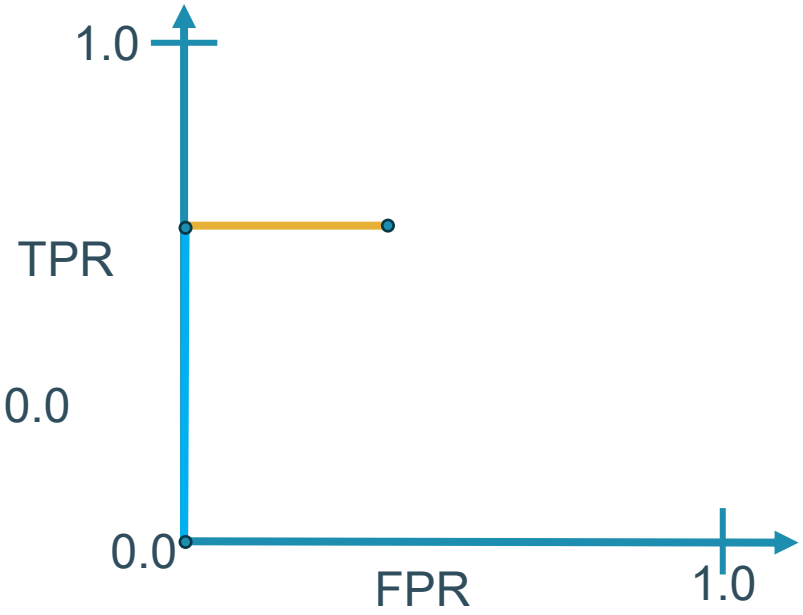
→

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.93 | + |
| 0.73 | + |
| 0.63 | - |
| 0.61 | - |
| 0.52 | + |
| 0.37 | - |
| 0.33 | + |
| 0.12 | - |
| 0.05 | - |

FPR= 0.0, TPR= 0.0

FPR= 0.0, TPR= 0.6

FPR= 0.4, TPR= 0.6



# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

→

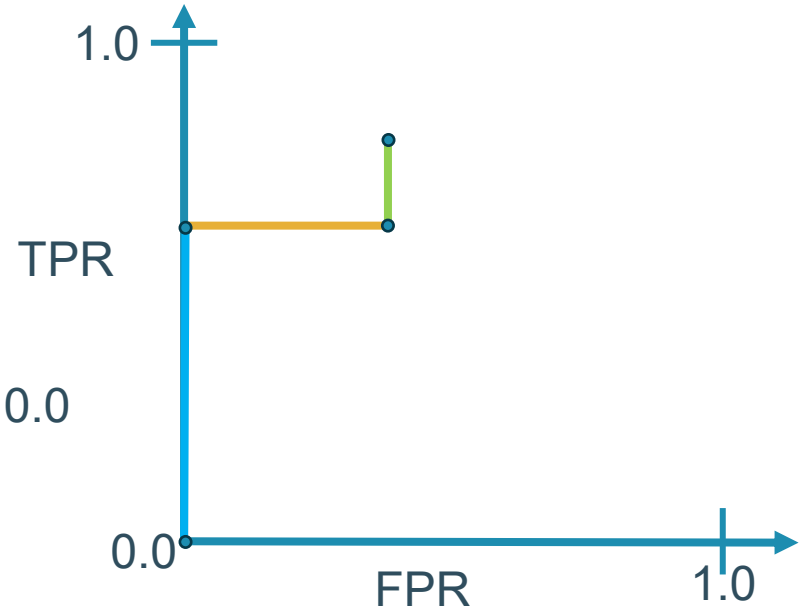
| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.93 | + |
| 0.73 | + |
| 0.63 | - |
| 0.61 | - |
| 0.52 | + |
| 0.37 | - |
| 0.33 | + |
| 0.12 | - |
| 0.05 | - |

FPR= 0.0, TPR= 0.0

FPR= 0.0, TPR= 0.6

FPR= 0.4, TPR= 0.6

FPR= 0.4, TPR= 0.8



# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

→

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.93 | + |
| 0.73 | + |
| 0.63 | - |
| 0.61 | - |
| 0.52 | + |
| 0.37 | - |
| 0.33 | + |
| 0.12 | - |
| 0.05 | - |

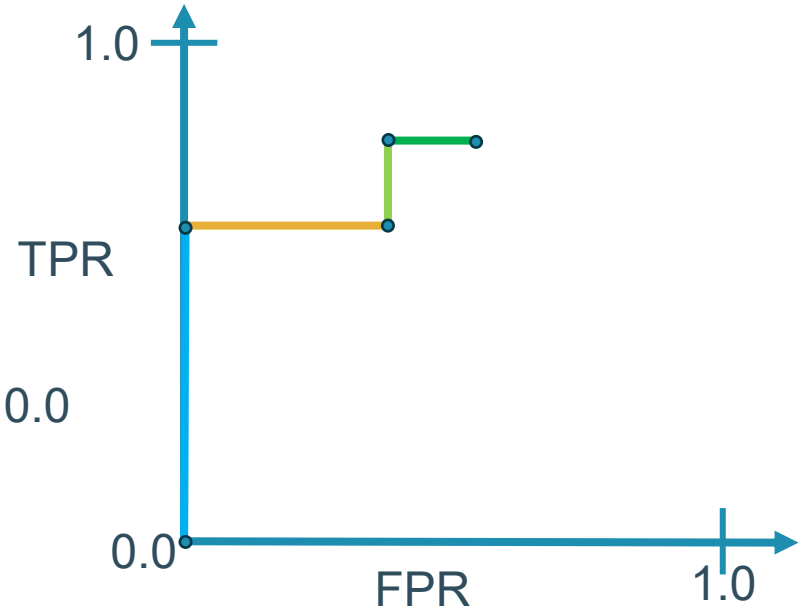
FPR= 0.0, TPR= 0.0

FPR= 0.0, TPR= 0.6

FPR= 0.4, TPR= 0.6

FPR= 0.4, TPR= 0.8

FPR= 0.6, TPR= 0.8



# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |

→

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.93 | + |
| 0.73 | + |
| 0.63 | - |
| 0.61 | - |
| 0.52 | + |
| 0.37 | - |
| 0.33 | + |
| 0.12 | - |
| 0.05 | - |

FPR= 0.0, TPR= 0.0

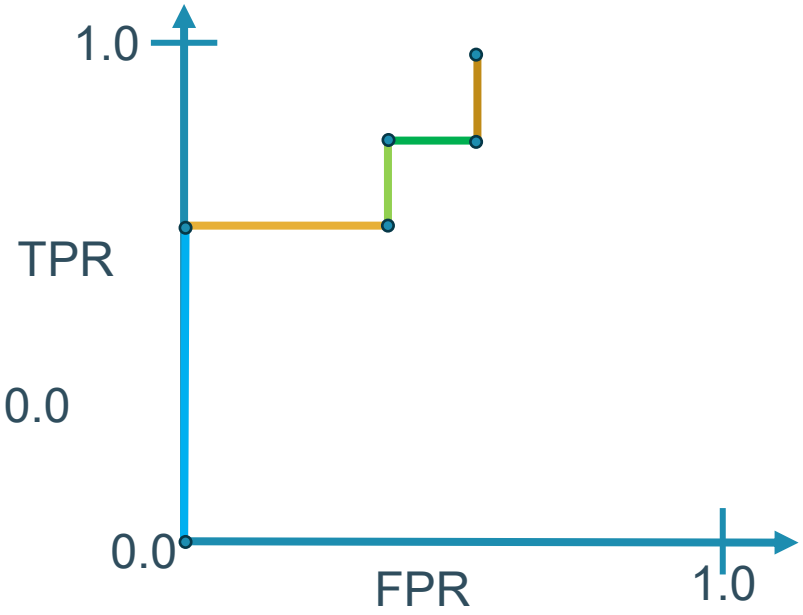
FPR= 0.0, TPR= 0.6

FPR= 0.4, TPR= 0.6

FPR= 0.4, TPR= 0.8

FPR= 0.6, TPR= 0.8

FPR= 0.6, TPR= 1.0

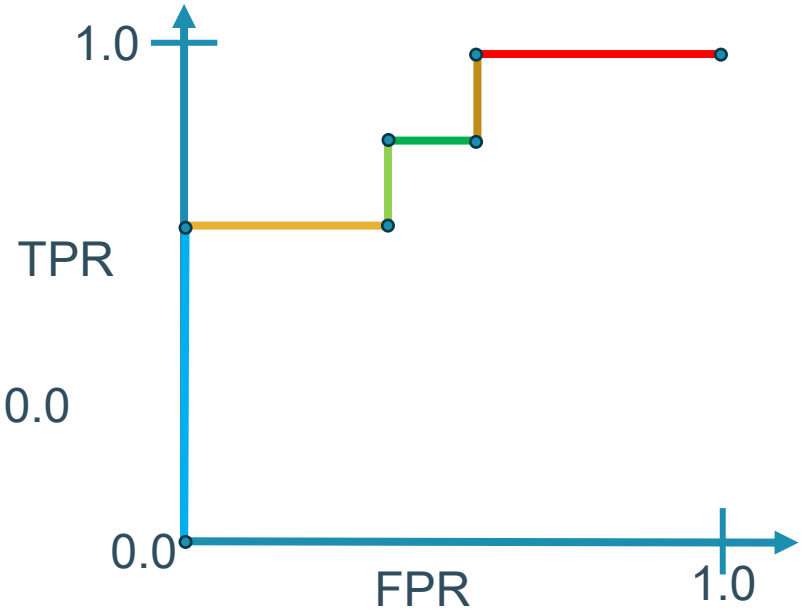


# ROC: example

| Predicted Score | Label |
|---|---|
| 0.95 | + |
| 0.12 | - |
| 0.61 | - |
| 0.37 | - |
| 0.73 | + |
| 0.63 | - |
| 0.93 | + |
| 0.33 | + |
| 0.05 | - |
| 0.52 | + |



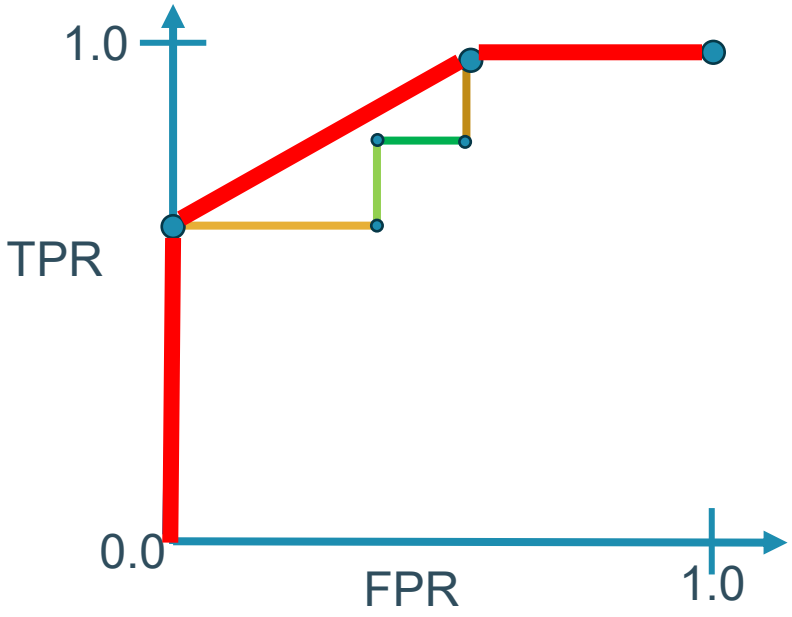
| Predicted Score | Label | |
|---|---|---|
| 0.95 | + | FPR= 0.0, TPR= 0.0 |
| 0.93 | + | |
| 0.73 | + | |
| 0.63 | - | FPR= 0.0, TPR= 0.6 |
| 0.61 | - | |
| 0.52 | + | FPR= 0.4, TPR= 0.6 |
| 0.37 | - | FPR= 0.4, TPR= 0.8 |
| 0.33 | + | FPR= 0.6, TPR= 0.8 |
| 0.12 | - | FPR= 0.6, TPR= 1.0 |
| 0.05 | - | FPR= 1.0, TPR= 1.0 |






KU LEUVEN

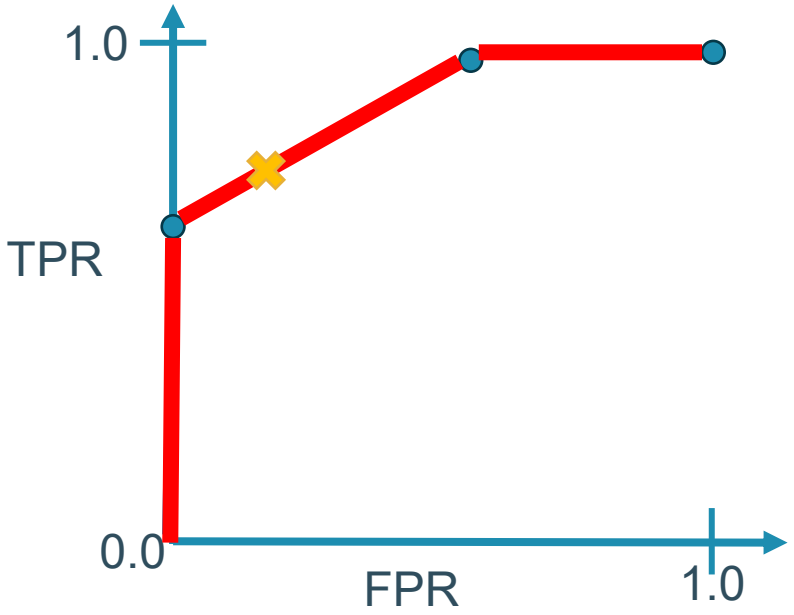
# Convex hull of a ROC curve




- Each point in the ROC space is a classifier
- Best classifier depends on operating conditions
  - Costs
  - Class skew
- Convex hull: Set of best classifiers under some operating condition
- Classifiers that are not on the hull → Never optimal under the considered linear cost settings.
- How do we get all the classifiers on the hull?
  → Interpolation

# Interpolation in ROC space

- Two (FPR, TPR) points
  - M1 (0.0, 0.6)
  - M2 (0.8,1.0)
- Goal: FPR=0.2 , TPR=0.7
- Flip biased coin
  - Select M1 with p=0.75
  - Select M2 with p=0.25
- Coin flipping allows linear interpolation



# Summarizing ROC curves

- Area under the ROC curve (AUC)
  - Wilcoxon-Mann-Whitney statistic:

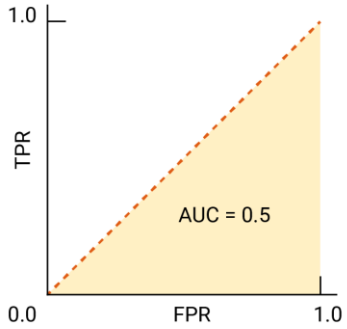
    *Probability that any random positive example is ranked ahead of any random negative example*

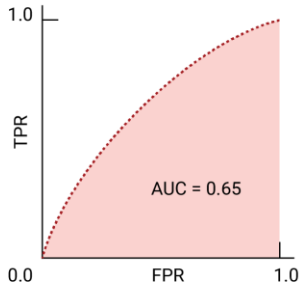
# Summarizing ROC curves

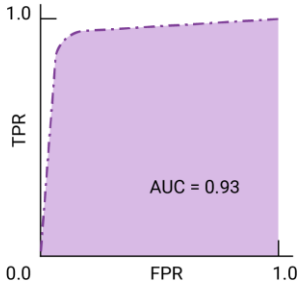
- Area under the ROC curve (AUC)
  - Wilcoxon-Mann-Whitney statistic:

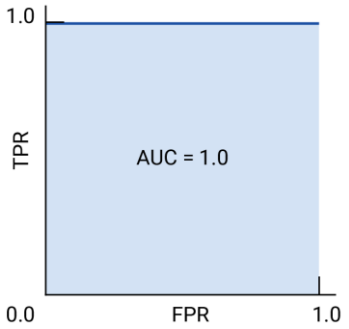
  *Probability that any random positive example is ranked ahead of any random negative example*

  *What are the ROC curves and AUC values for random guessing and a perfect model?*






KU LEUVEN

# ROC: class skew

- ROC can be misleading if:

  #positive examples << # negative examples

- ROC looks at fractions, not total numbers
  - If lots and lots of negative examples, then a very low FPR can still mean lots of FPs
  - E.g., Cancer diagnosis, search results, anomalies

- Solution: Precision and Recall

# Precision-Recall (PR) Curves

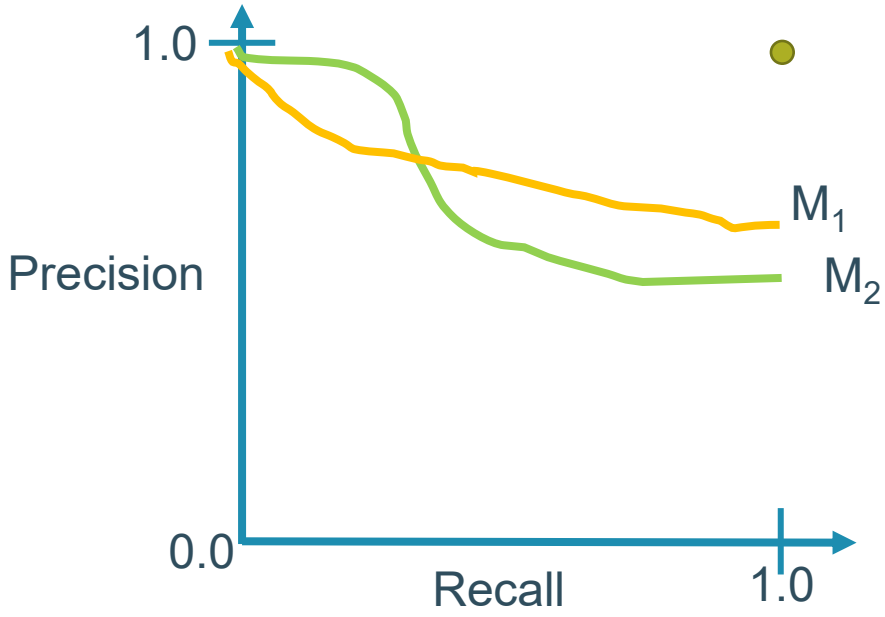
- Primarily arise out of the literature on information retrieval (web search)
- Useful when there are lots of negative examples, but you really care about the positive class (e.g. Cancer research)
- Arguable, these are the metrics you often want

# Precision-Recall (PR) Curves

- Primarily arise out of the literature on information retrieval (web search)
- Useful when there are lots of negative examples, but you really care about the positive class (e.g. Cancer research)
- Arguable, these are the metrics you often want
- PR curves plot
  - Recall = $\frac{TP}{TP+FN}$

    (How many of the positive examples are classified positive)
  - Precision= $\frac{TP}{TP+FP}$

    (How many of the classified positives are actually positives)



# Problem 1: Linear interpolation in PR space

– What happens when we interpolate? ([Goadrich, et al., ILP 2004])

| Data | | ROC | | PR | |
|---|---|---|---|---|---|
| TP | FP | TPR | FPR | Recall | Precision |
| 500 | 500 | | | | |
| | | | | | |
| | | | | | |



KU LEUVEN

# Problem 1: Linear interpolation in PR space

– What happens when we interpolate? ([Goadrich, et al., ILP 2004])

| Data | | ROC | | PR | |
|---|---|---|---|---|---|
| TP | FP | TPR | FPR | Recall | Precision |
| 500 | 500 | 0.50 | 0.06 | | |
| | | | | | |
| | | | | | |



KU LEUVEN

# Problem 1: Linear interpolation in PR space

– What happens when we interpolate? ([Goadrich, et al., ILP 2004])

| Data | | ROC | | PR | |
|---|---|---|---|---|---|
| TP | FP | TPR | FPR | Recall | Precision |
| 500 | 500 | 0.50 | 0.06 | 0.50 | 0.50 |
| | | | | | |
| | | | | | |



KU LEUVEN

# Problem 1: Linear interpolation in PR space

– What happens when we interpolate? ([Goadrich, et al., ILP 2004])

| Data | | ROC | | PR | |
|---|---|---|---|---|---|
| TP | FP | TPR | FPR | Recall | Precision |
| 500 | 500 | 0.50 | 0.06 | 0.50 | 0.50 |
| | | | | | |
| 1000 | 9000 | 1.00 | 1.00 | 1.00 | 0.10 |



KU LEUVEN

# Problem 1: Linear interpolation in PR space

– What happens when we interpolate? ([Goadrich, et al., ILP 2004])

| Data | | ROC | | PR | |
|---|---|---|---|---|---|
| TP | FP | TPR | FPR | Recall | Precision |
| 500 | 500 | 0.50 | 0.06 | 0.50 | 0.50 |
| 750 | 4750 | | | | |
| 1000 | 9000 | 1.00 | 1.00 | 1.00 | 0.10 |



KU LEUVEN

# Problem 1: Linear interpolation in PR space

– What happens when we interpolate? ([Goadrich, et al., ILP 2004])

| Data | | ROC | | PR | |
|---|---|---|---|---|---|
| TP | FP | TPR | FPR | Recall | Precision |
| 500 | 500 | 0.50 | 0.06 | 0.50 | 0.50 |
| 750 | 4750 | 0.75 | 0.53 | | |
| 1000 | 9000 | 1.00 | 1.00 | 1.00 | 0.10 |

# Problem 1: Linear interpolation in PR space

– What happens when we interpolate? ([Goadrich, et al., ILP 2004])

| Data | | ROC | | PR | |
|---|---|---|---|---|---|
| TP | FP | TPR | FPR | Recall | Precision |
| 500 | 500 | 0.50 | 0.06 | 0.50 | 0.50 |
| 750 | 4750 | 0.75 | 0.53 | 0.75 | 0.14 |
| 1000 | 9000 | 1.00 | 1.00 | 1.00 | 0.10 |

# Interpolation in PR space

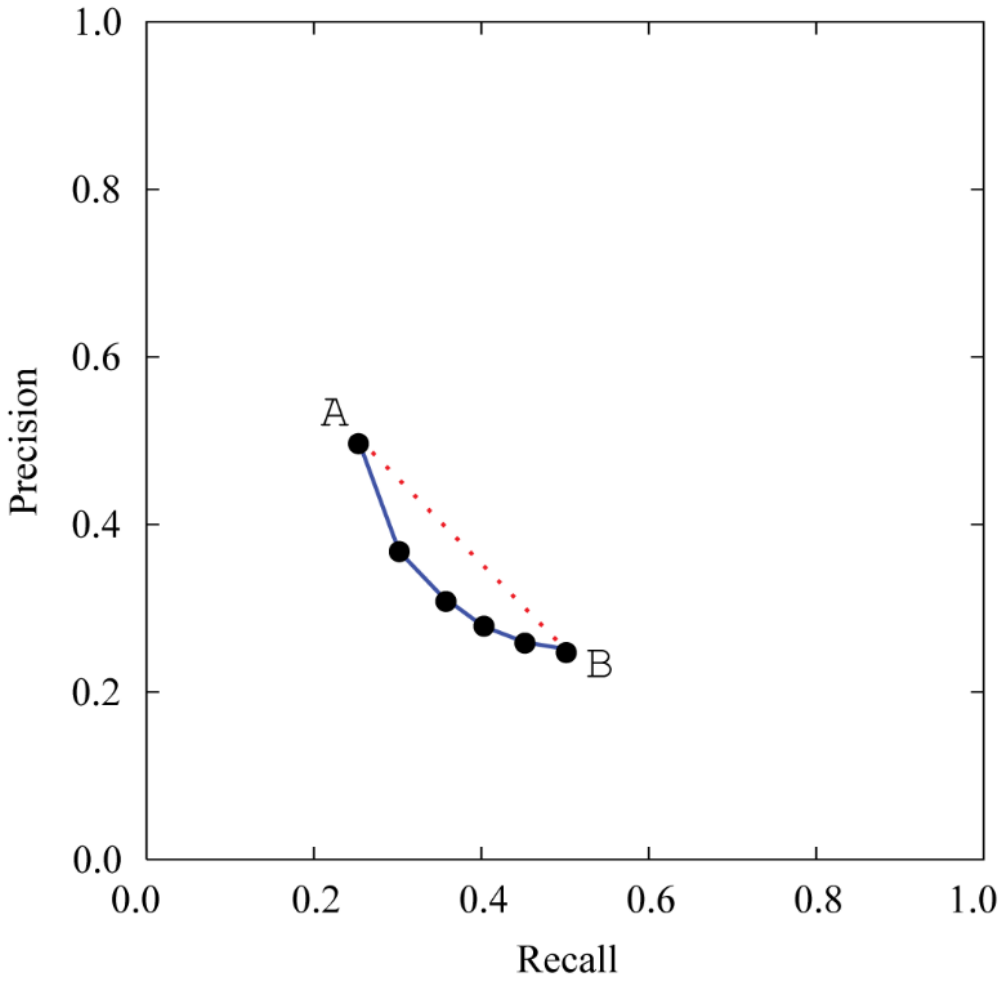
| | TP | FP | Recall | Precision |
|---|---|---|---|---|
| A | 5 | 5 | 0.25 | 0.5 |
| | 6 | 10 | 0.30 | 0.375 |
| | 7 | 15 | 0.35 | 0.318 |
| | 8 | 20 | 0.40 | 0.286 |
| | 9 | 25 | 0.45 | 0.265 |
| B | 10 | 30 | 0.50 | 0.25 |

Dataset with 20 positives and 2000 negatives

# Interpolation in PR space

| | TP | FP | Recall | Precision |
|---|---|---|---|---|
| A | 5 | 5 | 0.25 | 0.5 |
| | 6 | 10 | 0.30 | 0.375 |
| | 7 | 15 | 0.35 | 0.318 |
| | 8 | 20 | 0.40 | 0.286 |
| | 9 | 25 | 0.45 | 0.265 |
| B | 10 | 30 | 0.50 | 0.25 |

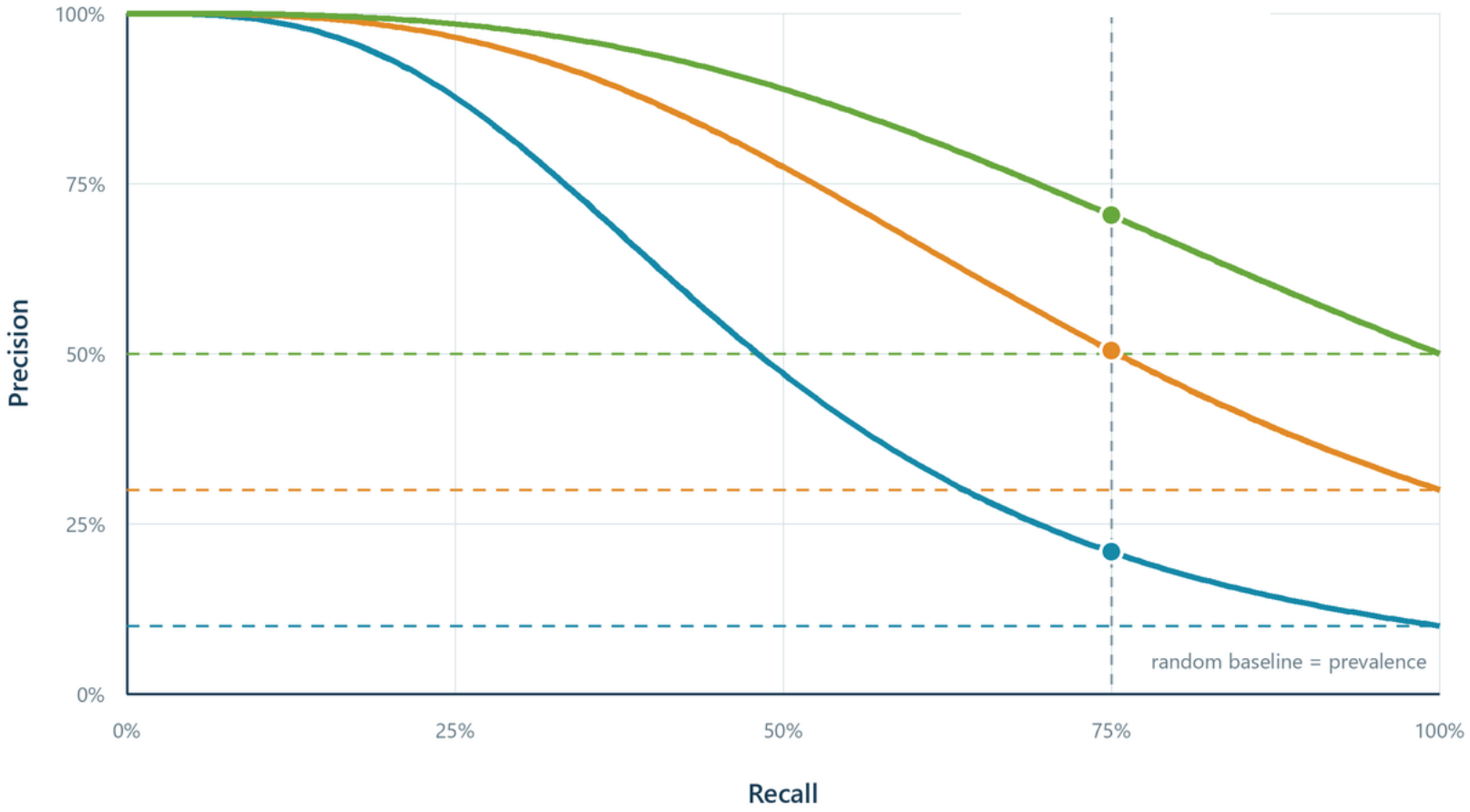
Dataset with 20 positives and 2000 negatives






KU LEUVEN

# Problem 2: Changes in skew




**At the same threshold**

Recall stays at 75%

- 10% positives
  Precision = 21%
- 30% positives
  Precision = 50%
- 50% positives
  Precision = 70%

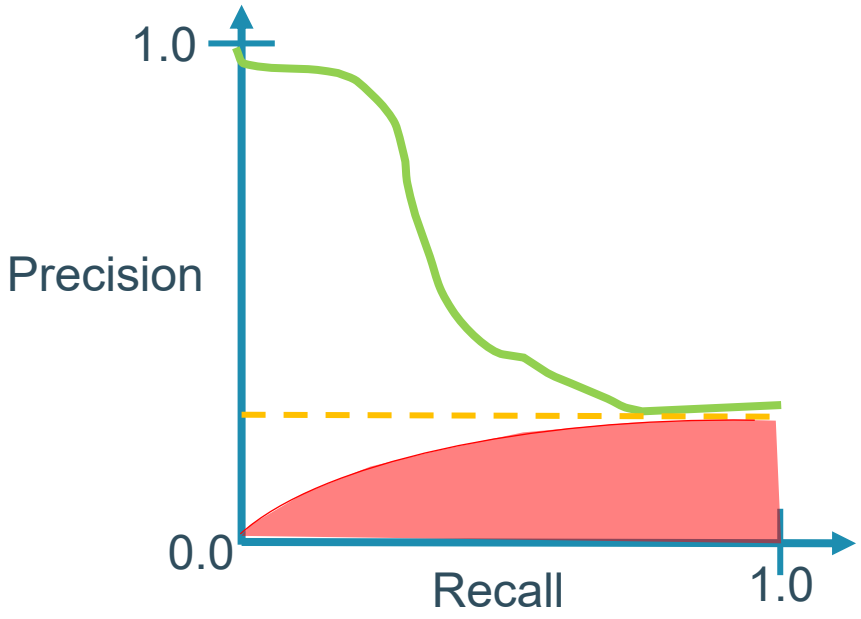
More positives among the same predicted positives → higher precision



KU LEUVEN

# Problem 3: Unachievable region

- In ROC space, all models are possible
- In PR space, it depends on the skew




Unachievable region: Not enough negative examples in the test set to construct the confusion matrix for these models

Only depends on the skew

# Problem 3: Unachievable region

- Key Insight: ROC and PR curves are constructed from a fixed dataset
- Implication: number of positive and negative examples are fixed
- Suppose: P=100, N=1000
  - Suppose i want Recall=0.9 and Precision=0.05
  - Recall = $\frac{TP}{TP+FN}$ ➔ **TP=90 ; FN=10**
  - Precision = $\frac{TP}{TP+FP}$ ➔ $0.05 = \frac{90}{90+FP}$ ➔ **FP=1710**

  1710 > 1000 ➔ Model not possible



KU LEUVEN

# Problem 4: comparison of 2 metrics

- Two models
  - M1: Prec=0.80  Rec=0.80
  - M2: Prec=0.60  Rec=0.90
- Which one is better?
  - Based on situation, but in general?

# Problem 4: comparison of 2 metrics

- Two models
  - M1: Prec=0.80  Rec=0.80
  - M2: Prec=0.60  Rec=0.90
- Which one is better?
  - Based on situation, but in general?
- E.g. F1-score combines precision and recall: $F1 = 2\frac{Prec \times Rec}{Prec + Rec} = \frac{2 \times TP}{2 \times TP + FP + FN}$

  F1(M1)=0.80 ; F1(M2)=0.72



KU LEUVEN

# How certain are we that these numbers are correct?

- All metrics are based on a finite set
  - How confident are we that it's correct?
- $\alpha\%$ Confidence intervals: if we repeatedly sampled datasets and constructed intervals this way, approximately $\alpha$ % of those intervals would contain the true value.
- For large samples, a good approximation is to assume error is normally distributed

$$Confidence\ interval\ for\ metric\ Me\text{:}\ \ CI_{Me} = Me \pm z\sqrt{\frac{Me \times (1 - Me)}{N_{Me}}}$$

| CI Width | 80% | 90% | 95% | 99% |
|---|---|---|---|---|
| z | 1.28 | 1.64 | 1.96 | 2.58 |

# How certain are we that these numbers are correct?

- All metrics are based on a finite set
  - How confident are we that it's correct?
- $\alpha\%$ Confidence intervals: if we repeatedly sampled datasets and constructed intervals this way, approximately $\alpha$ % of those intervals would contain the true value.
- For large samples, a good approximation is to assume error is normally distributed

$$Confidence\ interval\ for\ metric\ Me\text{:}\quad CI_{Me} = Me \pm z\sqrt{\frac{Me \times (1 - Me)}{N_{Me}}}$$

$$CI_{Prec} = Prec \pm z\sqrt{\frac{Prec \times (1 - Prec)}{TP + FP}} \qquad CI_{Rec} = Rec \pm z\sqrt{\frac{Rec \times (1 - Rec)}{TP + FN}}$$

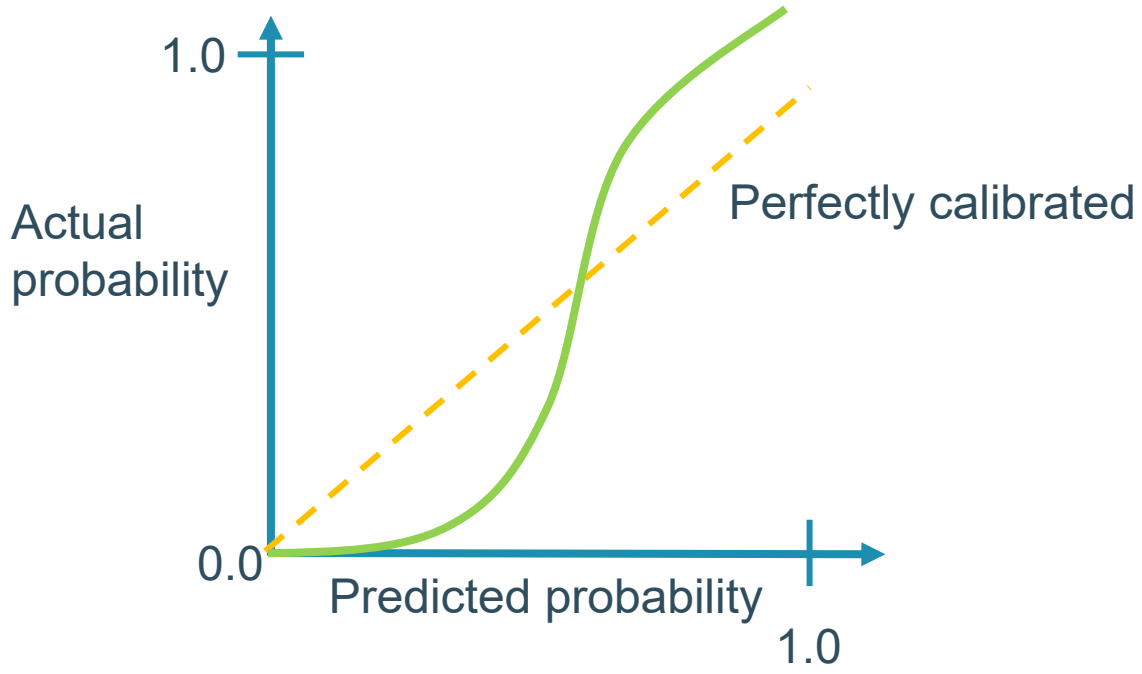
| CI Width | 80% | 90% | 95% | 99% |
|---|---|---|---|---|
| z | 1.28 | 1.64 | 1.96 | 2.58 |



KU LEUVEN

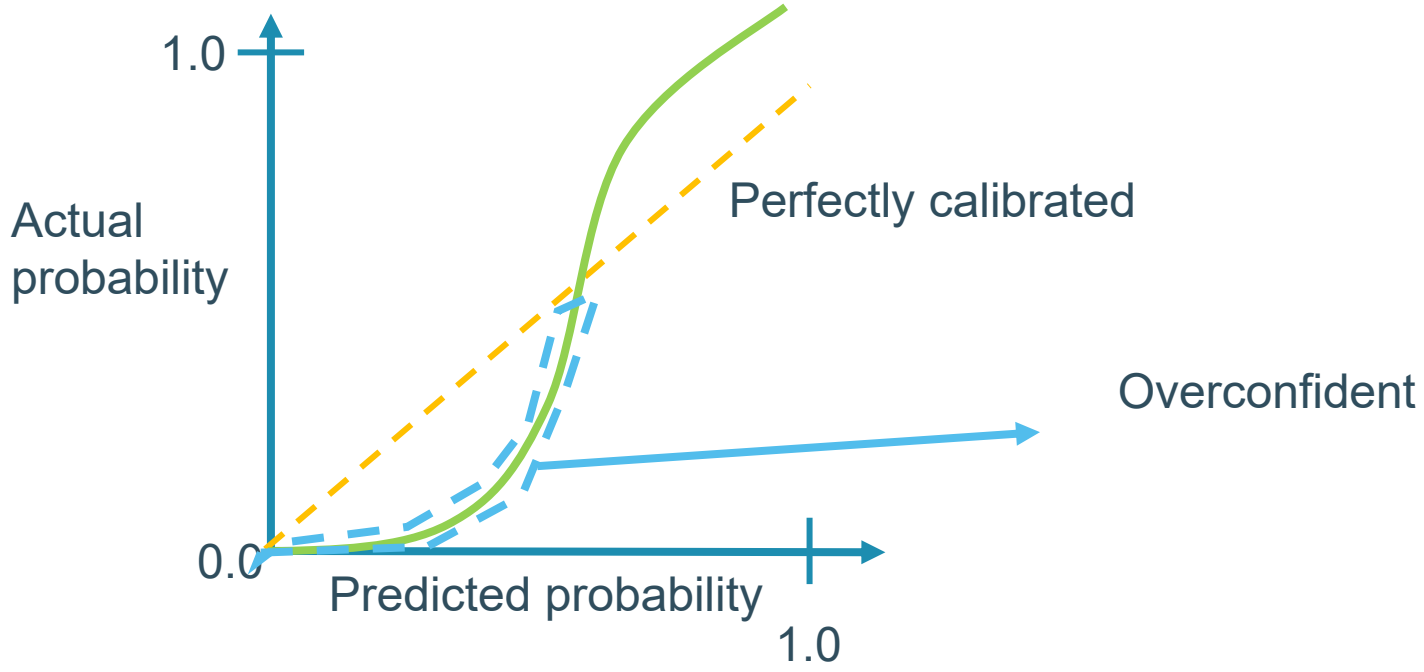
# Probability estimation

– Practice: Accurate probabilities are calibrated probabilities



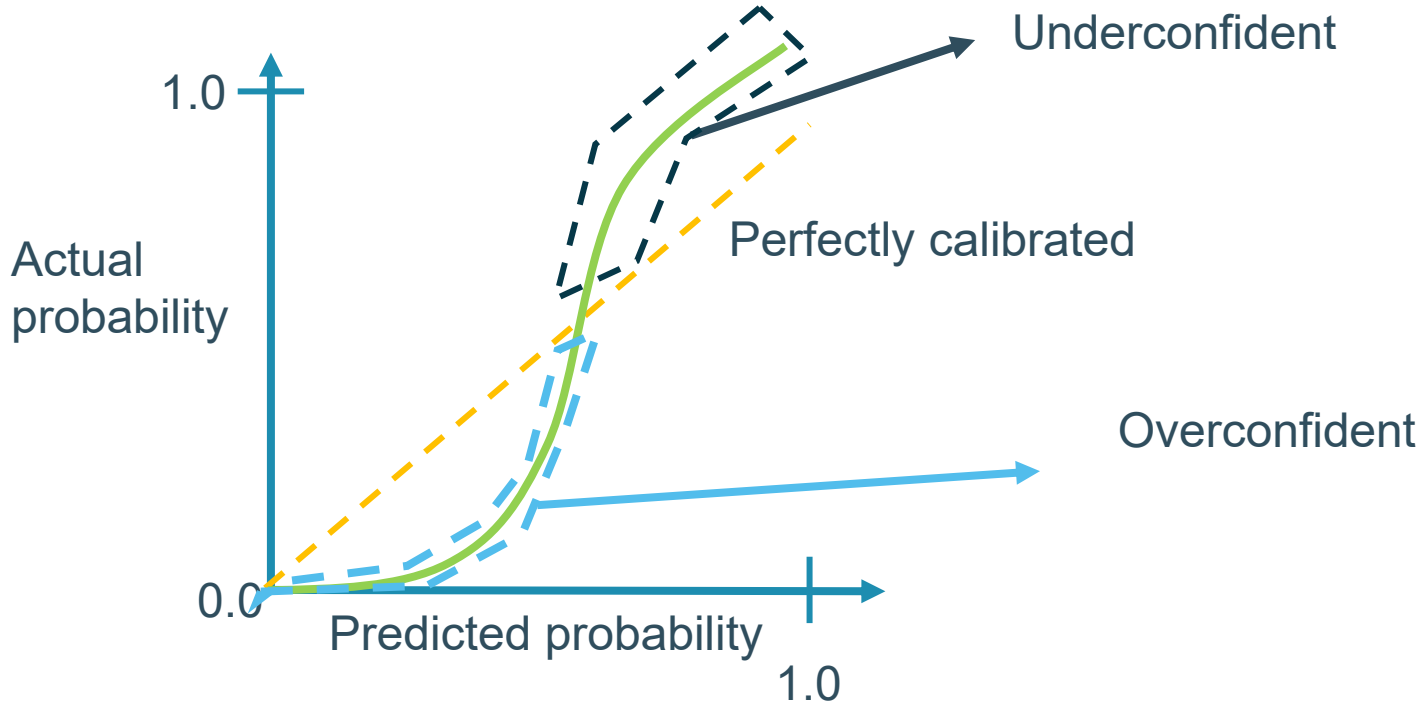
# Probability estimation

– Practice: Accurate probabilities are calibrated probabilities



# Probability estimation

– Practice: Accurate probabilities are calibrated probabilities



# Calibration

- Many classifiers are NOT well calibrated
- Can post-process scores to calibrate
  - Isotonic Regression
  - Beta calibration

- Brier score measures calibration for N examples and C classes

$$Brier\ Score = \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C}\left(y_{i,j} - p_{i,j}\right)^2$$

# Evaluating Regression models (real-valued outputs)

- Root mean squared error: $RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(f(x_i) - y_i)^2}$
- Mean absolute error: $MAE = \frac{1}{N}\sum_{i=1}^{N}|f(x_i) - y_i|$

- Relative Error = $\frac{\sum_{i=1}^{N}(f(x_i)-y_i)^2}{\sum_{i=1}^{N}(\bar{y}-y_i)^2}$

  1 = no better than always predicting the mean of the possible values

  0 = perfect prediction



KU LEUVEN

# Evaluation methodology



KU LEUVEN

# Suppose you want to deploy a model

- Predict the price of stocks
- Predict the risk of cancer
- Predict whether an email is spam or not
- Predict whether a song belongs to a playlist or not

# Suppose you want to deploy a model

- Predict the price of stocks
- Predict the risk of cancer
- Predict whether an email is spam or not
- Predict whether a song belongs to a playlist or not

**Question:** Independent of a metric, what is a key performance property that we'd need to actually be able to use these models?

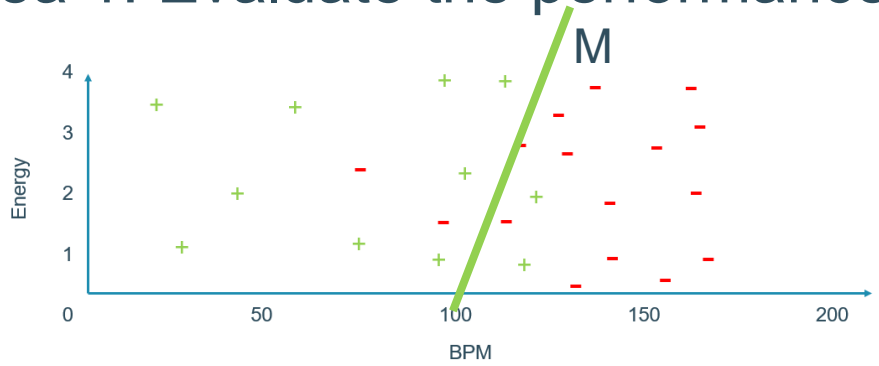
# Methodology: Generalization



KU LEUVEN

# Generalization

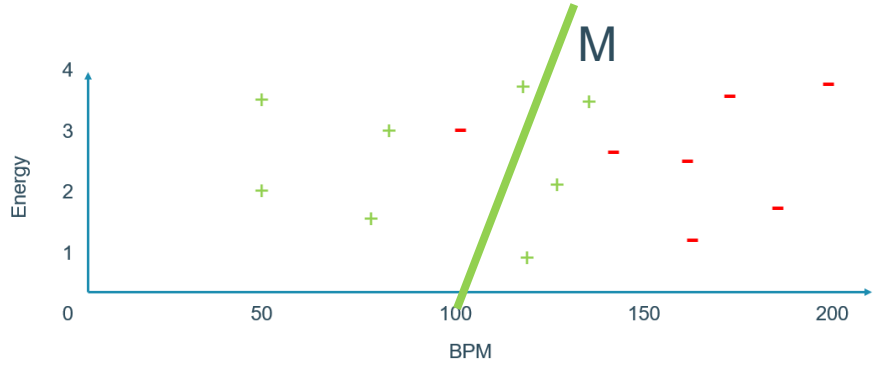
– Idea 1: Evaluate the performance on the data used to learn the model (training data)




$$F_1(M, train) = 0.818$$

$$Error\ rate(M, train) = 0.154$$

– Idea 2: use an independent test set consisting of instances that played no part in the formation of the classifier




$$F_1(M, test) = 0.714$$

$$Error\ rate(M, test) = 0.286$$



KU LEUVEN

# Solution 1: Split data into train and test data

- Dilemma: need both a large training set and a large test set
  - More training data → better classifier
  - More test data → better error estimate for generalization

# Solution 1: Split data into train and test data

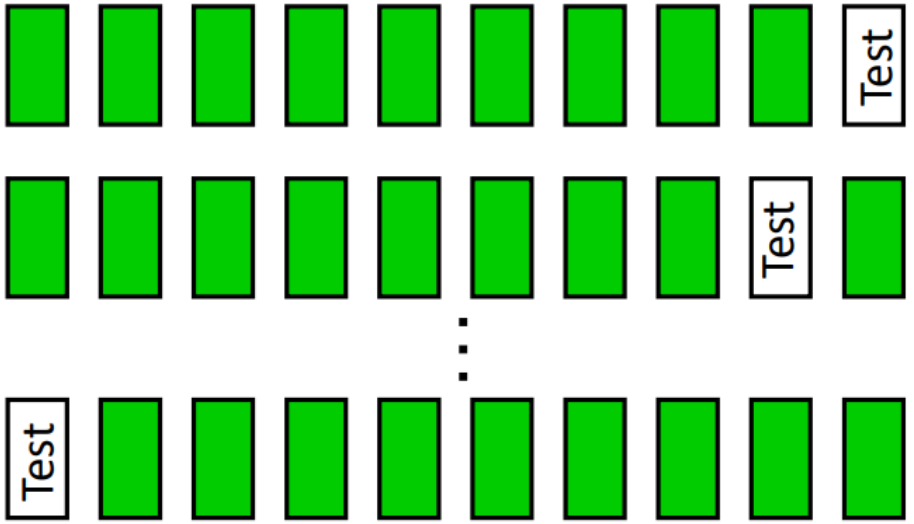
- Dilemma: need both a large training set and a large test set
  - More training data → better classifier
  - More test data → better error estimate for generalization
- Problem: (labeled) data is often limited and expensive to obtain
  - Research: open-source benchmark datasets, not method-specific
  - Industry: data initially doesn't immediately generate value/revenue, so gathering and labelling large datasets have low priority

# Solution 1: Split data into train and test data

- Dilemma: need both a large training set and a large test set
  - More training data → better classifier
  - More test data → better error estimate for generalization
- Problem: (labeled) data is often limited and expensive to obtain
  - Research: open-source benchmark datasets, not method-specific
  - Industry: data initially doesn't immediately generate value/revenue, so gathering and labelling large datasets have low priority
- Solution: use more sophisticated techniques
  - Can we mimic a large dataset with a small dataset?

# Solution 2: Cross validation

- Partition examples into *k* disjoint sets
- Create *k* training sets
  - Each set is union of all folds except one
  - So each set has $\frac{k-1}{k}$ of the original training data






KU LEUVEN

# Cross validation example

- Five-fold cross validation with 250 examples
- 50 examples per fold

| Training folds | Testing fold | Correct | Incorrect | Accuracy |
|---|---|---|---|---|
| 1 2 3 4 | 5 | 35 | 15 | 0.70 |
| 1 2 3 5 | 4 | 30 | 20 | 0.60 |
| 1 2 4 5 | 3 | 37 | 13 | 0.74 |
| 1 3 4 5 | 2 | 33 | 17 | 0.66 |
| 2 3 4 5 | 1 | 40 | 10 | 0.80 |

Overall accuracy = 175/250 = 0.70



KU LEUVEN

# Cross validation in practice

- Smaller data → More folds
- If you have very little data (<100 examples), you can use leave-one-out cross validation
  - Set number of folds to the number of examples
  - For *n* examples, train your classifier *n* times on *n-1* examples
  - Involves no random subsampling, every instance is used *n-1* times as training data and once as test data
- Typically, people use 5-fold or 10-fold cross validation
- With large datasets (>1 million examples), a single split into train-test is OK!

# One potential problem

- Representativeness problem:
  - All examples from a whole (infrequent) class might be missing in the training data
  - Error estimate is then meaningless

# One potential problem

- Representativeness problem:
  - All examples from a whole (infrequent) class might be missing in the training data
  - Error estimate is then meaningless
- Solution: Stratification
  - Ensure that each class is represented with its original class proportion in both train and test sets
  - Error estimates bear meaning
  - Variance in estimates will reduce

# Big Pitfall: is test data representative of real data?

- Possible that held aside test data is not representative of new data gathered as test data!
- Even with correct methodology:
  - Positive and negative examples collected differently
  - Features acquired differently for different classes
  - The split should mimic deployment and keep dependent observations together!
- Examples:
  - Cancer samples processed by different people or on different days than normal samples
  - Song belongs to a playlist on one day, not on the other

# Notes on cross-validation

- Data is stratified before the cross validation is performed
- All data is used for testing!
- The predictions over the test folds are concatenated or the error estimates are averaged to yield an overall error estimate
- Because we use multiple training sets in Cross-Validation, we are evaluating a learning method as opposed to an individual learned model
- Did I tweak my algorithm a million times until I get good results?
  - Solution: Use one or two datasets for development, then expand evaluation

# Methodology: Manipulating the feature space



KU LEUVEN

# Manipulating the feature space

- This can be for a variety of reasons:
  - Tools may support only real-valued features or only discrete features
  - Can improve computational efficiency
    - String comparisons are slower than integer comparisons
    - Fewer options to try = faster run time
  - Can improve accuracy
- Two categories of operations
  - Manipulating the values a feature can take on
  - Removing features

# Transforming ordinals and nominals

- Convert ordinal to numeric which enables using > or <
  - E.g. Tempo={Very slow, slow, quite slow, normal, quite fast, fast, very fast}
    →Very slow=1; slow=2; quite slow=3; normal=4, …
- One-hot encoding of nominal features
  - One binary indicator per value

| Genre |
|---|
| Jazz |
| Punk |
| Opera |



| Genre=Jazz | Genre=Punk | Genre=Opera |
|---|---|---|
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 1 |

Note: converting to integers incorrectly implies there's an ordering or relation between categories when there isn't

# Handling features with many possible values

- Group values into categories by hand
  - Map e.g. US states to regions; subgenres into genres; years into decades
  - If many infrequent values, group them (e.g., new category “other”)
- Target encoding: Replace each feature value with a statistic derived from the target
  - Real: e.g. Average of target for each value
  - Discrete: e.g. Conditional probability of target for each value

| Artist | Plays | Skipped | Target Encoding |
|---|---|---|---|
| ABBA | 100.000 | 20.000 | 0.20 |
| Green Day | 50.000 | 20.000 | 0.40 |
| Eminem | 75.000 | 25.000 | 0.33 |
| Skrillex | 33.000 | 27.000 | 0.81 |

# Handling features with many possible values

- Group values into categories by hand
  - Map e.g. US states to regions; subgenres into genres; years into decennia
  - If many infrequent values, group them (e.g., new category “other”)
- Target encoding: Replace each feature value with a statistic derived about the target
  - Real: e.g. Average of target for each value
  - Discrete: e.g. Conditional probability of target for each value

| Artist | Plays | Skipped | Target Encoding |
|---|---|---|---|
| ABBA | 100.000 | 20.000 | 0.20 |
| Green Day | 50.000 | 20.000 | 0.40 |
| Eminem | 75.000 | 25.000 | 0.33 |
| Skrillex | 33.000 | 27.000 | 0.81 |

Warning: in k-fold cross validation, it has to be done per fold

# Continuous to Discrete: Binning

– BPM values: 50 70 80 100 110 120 130 140 160 180 190 240

Equal Width



[50,100) [100,150) [150,200) [200,250)

Equal Height



[50,90) [90,130) [130,170) [170,250)



KU LEUVEN

# Feature selection

- Feature selection: Remove features prior to learning
- Why?
  - Easier learning: Fewer parameters (good if |Features| ≫|Data|)
  - More interpretable models: Gain domain insights
  - Improve efficiency: Fewer features = less memory / runtime
  - Better performance: Irrelevant attributes degrade performance
- How?
  - Filter-based approaches
  - Wrapper-based approaches

# Filter-based feature selection

- Idea: Measure each feature's usefulness in isolation (i.e., independent of other features)
- Pro: Scales to large feature sets or large data sets
- Cons:
  - Misses feature interactions
  - May select redundant features



# Two filter-based approaches

- Metric that computes a “correlation”
  - Information gain with S the data and A the feature:

$$Gain(S, A) = Entropy(S) - \sum_{v \in values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

  - Pearson correlation coefficient with $f_i$ the $i^{th}$ feature and y the target:

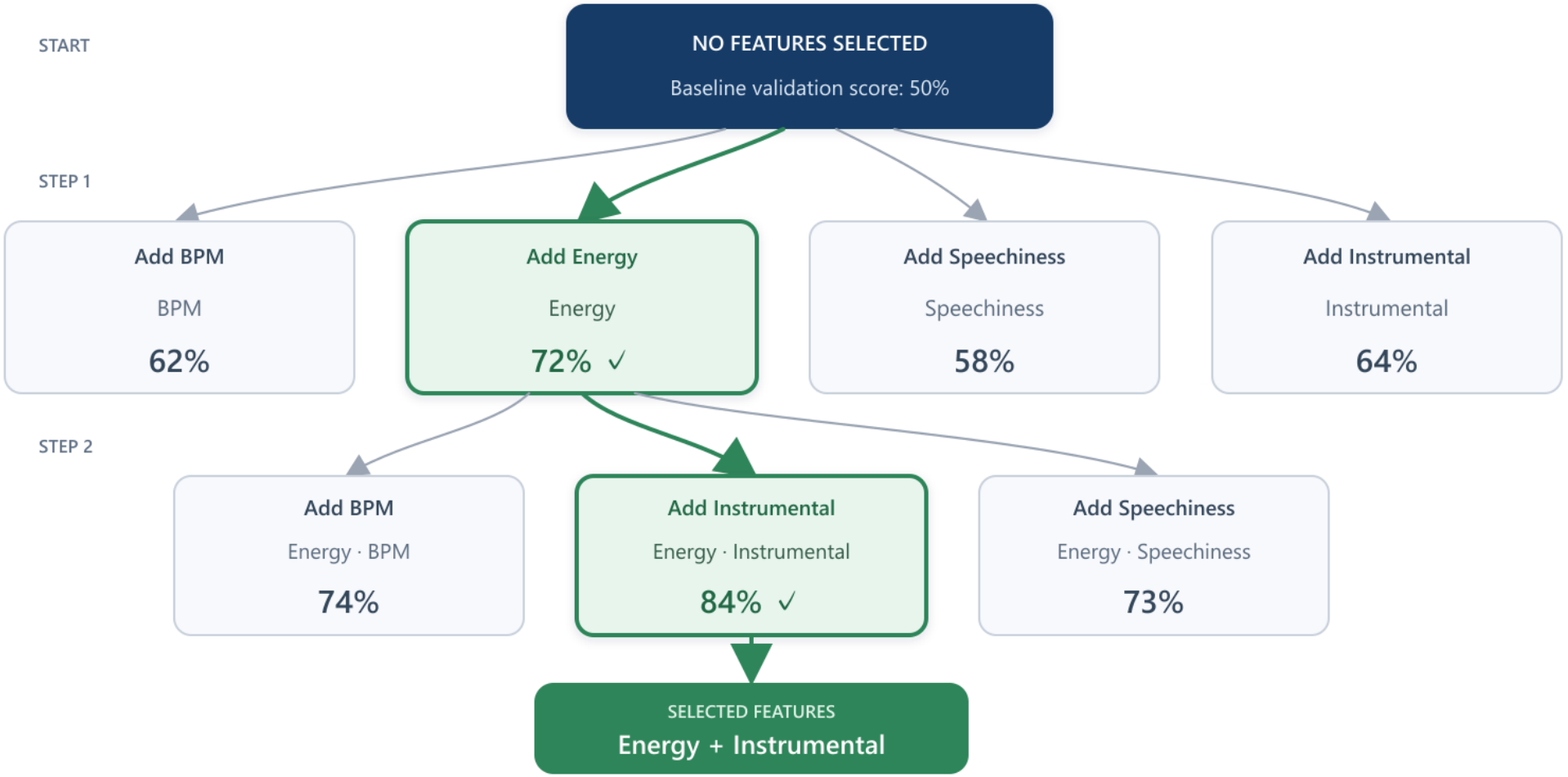
$$\mathrm{R}(f_i, y) = \frac{\mathrm{cov}(f_i, y)}{\sqrt{\mathrm{var}(f_i)\,\mathrm{var}(y)}}$$

- Single-variable classifier: Select based on predictive performance
  - Build classifier with just one variable
    - Discrete: Decision stump
    - Continuous: Threshold the feature’s value
  - Measure performance

# Wrapper-Based feature selection

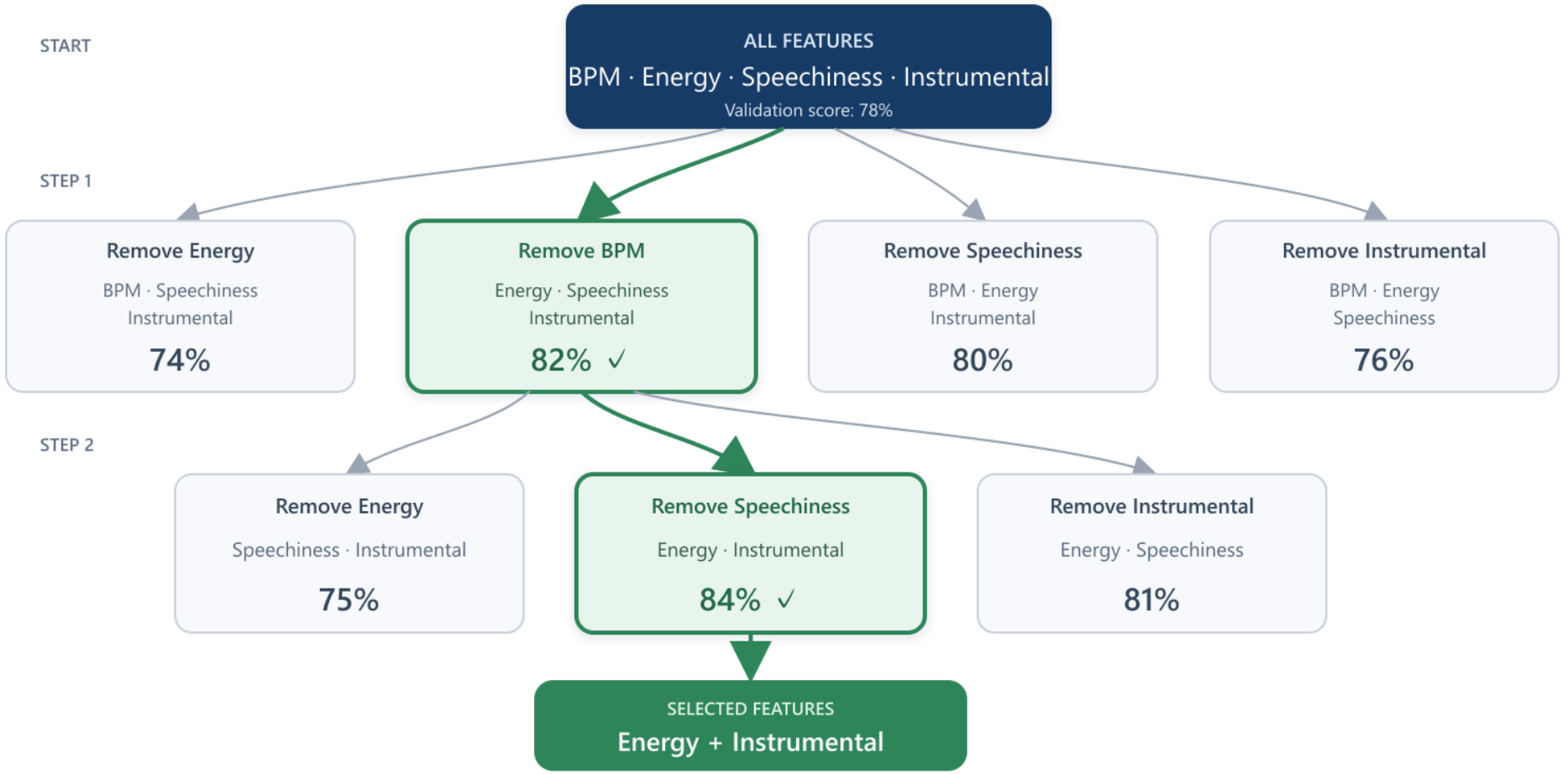
- Feature selection = search
- State = set of features
- Start state
  - Forward: No features
  - Backward: All features
- Operators:
  - Forward: Add a feature
  - Backward: Remove a feature
- Scoring function: Model's performance when using the state's feature set

# Forward wrapper-based feature selection




Illustrative validation scores

# Backward wrapper-based feature selection




Illustrative validation scores

# Forward vs backward

- Forward
  - Faster in early steps because fewer features to test
  - Fast for choosing a small subset of the features
  - Misses features whose usefulness requires other features (feature synergy)
- Backward
  - Fast for choosing all but a small subset of the features
  - Preserves features whose usefulness requires other features (e.g., area requires both length & width)

# Methodology: configuring the pipeline



KU LEUVEN

# Practical questions

- How should I set the value of k in kNN?
- How many trees should I use in my ensemble?
- How many features should I use?

- Idea 1: evaluate all choices on the training data, pick the best one
  - Problem: How to avoid overfitting?

# Tuning: subdivide the data

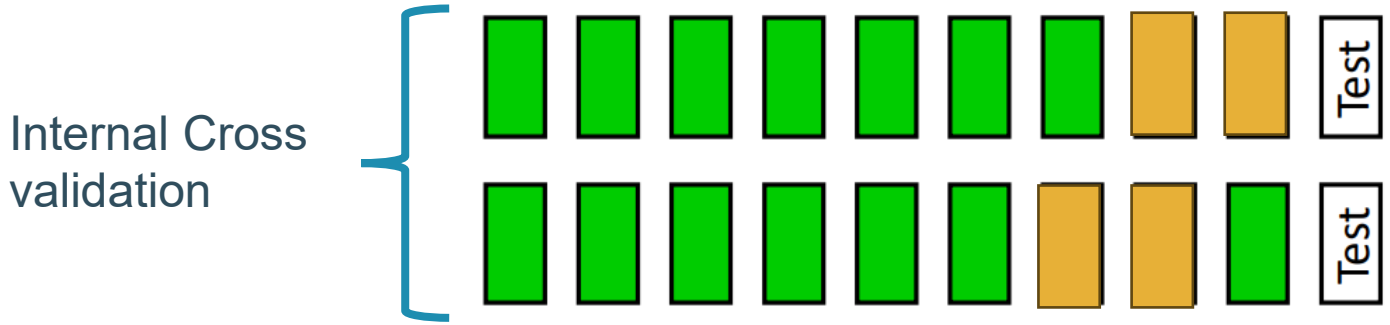
- Add an extra split: Training – **Validation** – Test
- For each considered algorithm configuration (e.g., number of trees in the ensemble, with/without feature selection):
  - Learn model on train
  - Evaluate model on validation
- Pick the model with the best performance on the validation set: Compute and report its test set performance

# Internal or Nested Cross Validation

- Single validation set can be unreliable
  - Do cross validation on the training data
  - Pick setting with best average performance
  - Learn model with chosen configuration using all training data

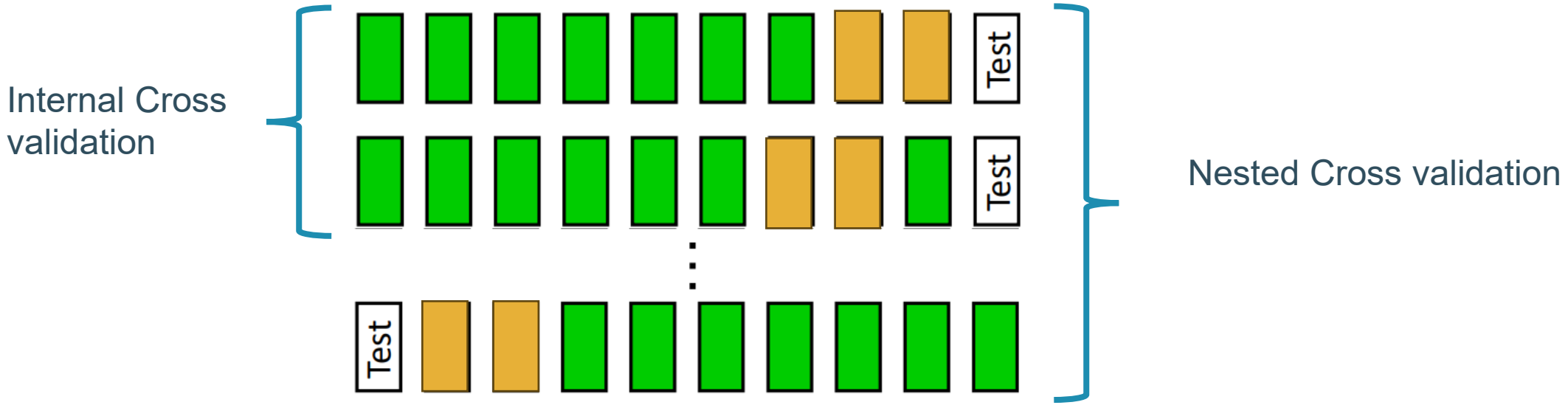
# Internal or Nested Cross Validation

- Single validation set can be unreliable
  - Do cross validation on the training data
  - Pick setting with best average performance
  - Learn model with chosen configuration using all training data



# Internal or Nested Cross Validation

- Single validation set can be unreliable
  - Do cross validation on the training data
  - Pick setting with best average performance
  - Learn model with chosen configuration using all training data



# Hyperparameter Search

- Problem: Often have to tune multiple parameters:
  - E.g. Random forests: number of trees, depth of trees, width of trees, …

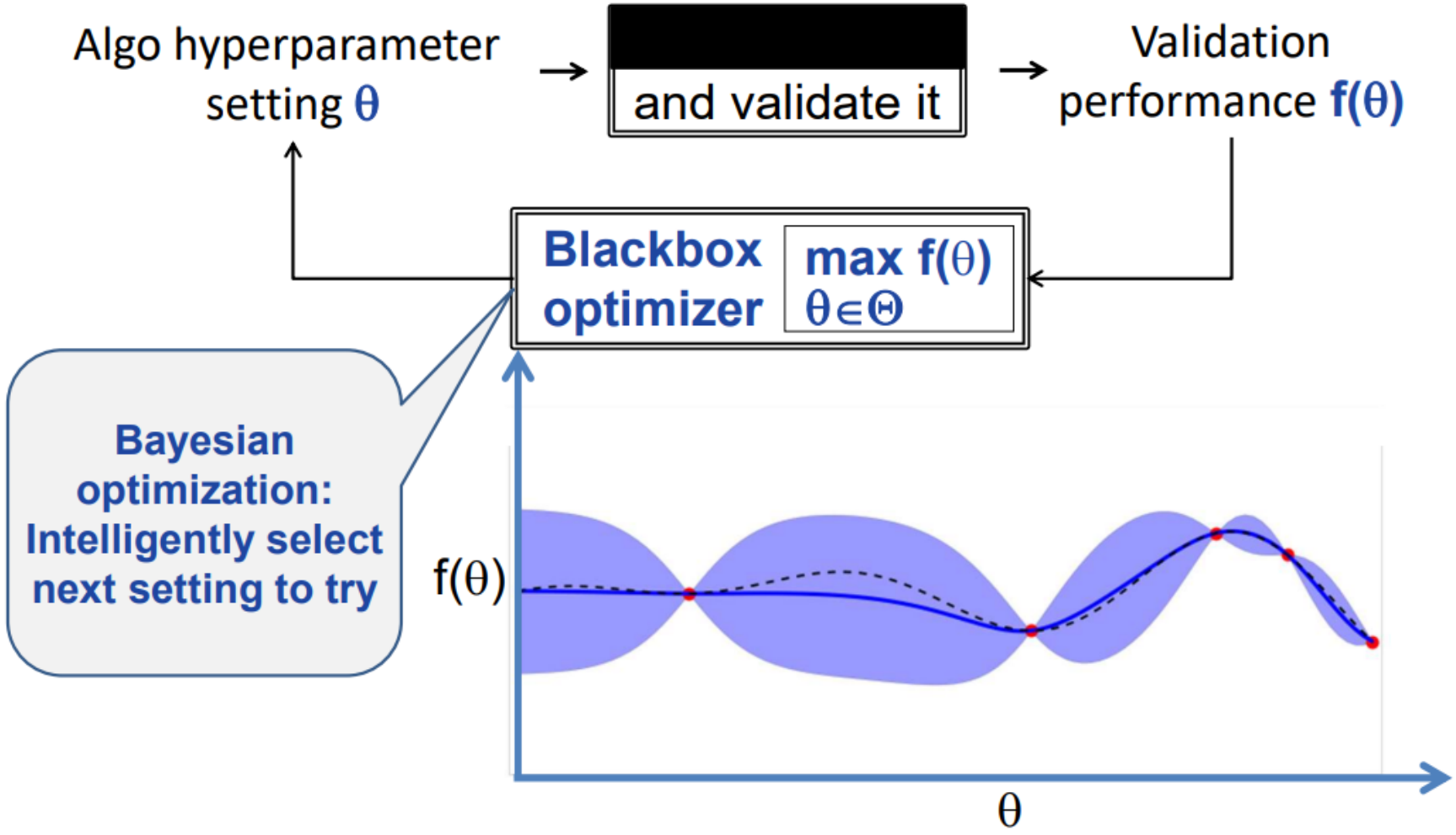
- Approach: Treat as a search problem
  - Grid search: Try all possible values
  - Random search: Sample possible values
    - is often quite effective (why?)

# Hyperparameter Search

- Problem: Often have to tune multiple parameters:
  - E.g. Random forests: number of trees, depth of trees, width of trees, …

- Approach: Treat as a search problem
  - Grid search: Try all possible values
  - Random search: Sample possible values
    - is often quite effective (why?) → Random search works well because usually only a few hyperparameters strongly affect performance, so it tests more distinct values of those parameters than a grid with the same budget.

# Automated Hyperparameter Tuning: -AutoWeka or auto-sklearn [from hutter]



# What to tune

- Decision trees
  - Minimum number of examples in leaf
  - Depth of tree
- Ensembles: Size of ensemble
- kNN: Value of k
- Random forest: max_features, number of trees, …
- Models: Linear and logistic regression
  - Regularization (L2, L1,…)
  - Regularization parameter λ
- With or without feature selection

# Values to be considered

- Need a reasonable spread of values:
  - Good: t = 50, 100, 500, 1000
  - Bad: t = 40, 45, 50, 55, 60
  - Good: k = 1, 5, 15, 25
  - Bad: k = 1, 3, 5
  - Good: $\lambda$ = 0.01, 0.05, 0.1, 0.5, …
  - Bad : $\lambda$ = 0.01, 0.02, 0.03
- Reasoning: Spread between values is important
- **Idea:** Use log-spaced values for scale parameters and choose bounds based on plausible values and computational budget.

# Leakage

- Information from outside the training set leaks into the training data (i.e., test set is no longer independent)
- Happens more often than you think
  - Common errors:
    - Global preprocessing, e.g., feature selection or target encoding before cross validation. Must repeat entire data processing pipeline on every fold of CV using only that fold's TRAINING DATA
    - Temporal dependencies: Use future data to predict the past
    - Incorrect or misaligned time stamps
    - Multiple examples for each entity: Must do cross validation on the subject level and not the example level

# Questions?



KU LEUVEN